

Chaos Engineering with Krkn

Version 1.0.0, 2026-08-10

Home

Introduction

Course Title: Chaos Engineering with Krkn

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Chaos engineering methodology
- Turn risk into a testable experiment
- Krkn Operator fundamentals
- Run and interpret one chaos experiment
- Build workflows with Chaos Studio
- Measure resilience and establish a baseline
- Operationalize resilience validation
- Hands-on Labs

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Chaos engineering methodology

Chaos engineering methodology

Chaos engineering is a proactive approach to identifying system weaknesses by injecting controlled disruptions. The methodology focuses on validating system resilience before failures occur in production environments.

- **Lifecycle Management:** Follows a structured process from hypothesis generation and steady-state definition to disruption execution and recovery validation.
- **Proactive Strategy:** Shifts from reactive incident response to proactive identification of failure modes, ensuring systems can withstand stress.
- **Performance Evaluation:** Measures key metrics—such as latency, throughput, and error rates—against established Service Level Objectives (SLOs) during disruption.
- **Safety and Control:** Prioritizes blast radius management and rollback planning to minimize impact on users, whether testing in staging or production environments.
- **Observability:** Relies on robust monitoring to analyze component behavior and verify automated recovery mechanisms during and after chaos injection.

Understanding the chaos engineering lifecycle

Understanding the chaos engineering lifecycle

Chaos engineering is not simply "breaking things." It is a disciplined, scientific approach to identifying weaknesses in distributed systems before they manifest as outages. By adopting a formal lifecycle, teams can transition from reactive firefighting to proactive system hardening.

The Chaos Engineering Lifecycle

The lifecycle of a chaos experiment consists of five iterative phases designed to ensure safety, repeatability, and meaningful data collection.

```
@startuml
[*] --> DefineSteadyState
DefineSteadyState --> Hypothesis
Hypothesis --> RunExperiment
RunExperiment --> AnalyzeResults
AnalyzeResults --> Remediate
Remediate --> [*]
@enduml
```

1. Define Steady State

Before introducing disruption, you must quantify "normal." A steady state is defined by a set of measurable metrics (KPIs) that represent the healthy operation of your service. * **Metrics:** Identify

latency, throughput, error rates, and resource utilization. * **Baseline:** Establish these values during a period of typical traffic to ensure the experiment is measured against an accurate reference point.

2. Hypothesis Generation

Translate a potential risk into a testable experiment. * **The "What if":** Ask, "What happens to our user login service if the underlying database cluster experiences a node failure?" * **Expected Outcome:** Predict the behavior. **Example:** "If the database primary fails, the system should trigger a failover within 30 seconds with no data loss."

3. Running the Experiment

Execute the disruption within a controlled environment, adhering to the principles of **Blast Radius management**. * **Targeting:** Specify the cluster, namespace, or specific set of pods. * **Observability:** Ensure monitoring tools (e.g., Prometheus/Grafana) are capturing high-resolution data during the event. * **Stop Conditions:** Pre-define automated "kill switches" that immediately halt the experiment if metrics cross a critical safety threshold.

4. Analysis and Comparison

Compare the results of the experiment against your baseline steady state. * **Validation:** Did the system recover as predicted? * **Gap Identification:** Did the failure propagate to unexpected components? Was there a "cascading failure" where one service outage took down dependent services?

5. Remediation and Iteration

Use the insights gathered to improve system resiliency. * **Improvements:** This might involve tuning Kubernetes readiness probes, adjusting circuit breaker patterns, or increasing replica counts. * **Repeat:** Once changes are implemented, run the experiment again to verify that the remediation was successful and did not introduce new risks.

Proactive vs. Reactive Testing

Strategy	Focus	Outcome		:---	:---	:---		Reactive	Responding to outages after they occur.
	Higher Mean Time to Recovery (MTTR); unpredictable downtime.							Proactive	Injecting faults to discover hidden weaknesses.
									Reduced risk of production failure; improved system architecture.



In the context of Krkn, we aim for **proactive** testing by automating these lifecycle stages. By mapping specific failure modes—such as pod deletions or network latency—to your service risks, Krkn allows you to transform abstract concerns into repeatable, quantifiable experiments.

Key Principles for Success

- **Manage the Blast Radius:** Always start with a small, contained blast radius before expanding to larger components.
- **Automation:** To make chaos engineering part of your culture, these lifecycle stages should eventually be integrated into your CI/CD pipelines to ensure that every code deployment is

inherently resilient.

- **Observability is Non-negotiable:** Without clear metrics, an experiment is just an outage. Ensure your observability stack is configured to capture the moment of disruption and the subsequent recovery phase.

Proactive versus reactive testing strategies

Proactive versus reactive testing strategies

In the context of Chaos Engineering with Krkn, moving from reactive troubleshooting to proactive resilience testing is a fundamental shift in how we manage system reliability. Understanding the distinction between these strategies is essential for reducing Mean Time To Repair (MTTR) and hardening your infrastructure before failures occur in production.

The Reactive Approach

Reactive testing is often described as "firefighting." It occurs after an incident has already compromised system availability.

- **Characteristics:** Triggered by production outages, customer reports, or unexpected performance degradation.
- **The Workflow:** Alert triggers → Investigation → Root Cause Analysis (RCA) → Patching.
- **Limitations:** By the time a reactive team responds, the business impact—such as downtime, data loss, or service-level objective (SLO) violations—has already occurred. Reliance solely on reactive testing leaves "unknown unknowns" in the architecture, where hidden single points of failure remain undiscovered until they inevitably break.

The Proactive Approach (Chaos Engineering)

Proactive testing, specifically through Chaos Engineering with Krkn, involves the deliberate injection of failures into a controlled environment to observe how the system handles turbulence.

- **Characteristics:** Deliberate, scheduled, and controlled disruption of services, network, or infrastructure components.
- **The Workflow:** Hypothesis generation → Blast radius definition → Failure injection → Monitoring steady state → Recovery validation.
- **Benefits:** Proactive testing exposes how your observability stack (alerts and monitoring) performs **under pressure**. It allows SRE teams to validate that automated recovery mechanisms—such as pod restarts, node autoscaling, or failover logic—actually function as designed.

Comparing Testing Strategies

Strategy	Focus	Outcome
Reactive	Responding to failures after they occur.	Minimizing the duration of downtime after an impact.

Strategy	Focus	Outcome
Proactive	Identifying weaknesses before they manifest as incidents.	Reducing the likelihood of impact and improving system hardening.

Best Practices for Transitioning

To move toward a proactive model using Krkn, implement the following operational standards:

1. **Define Steady State:** Before injecting chaos, define what "normal" looks like. Use baseline performance data (CPU, memory, latency, and throughput) as your reference point.
2. **Ensure Observability:** A proactive test is useless if you cannot measure the impact. Ensure that alerts are tuned for the expected disruption and that they include appropriate severity levels and SOP documentation.
3. **Minimize Blast Radius:** Start small. Use Krkn to target specific namespaces or non-critical worker nodes before expanding the blast radius to core platform services.
4. **Integrate Recovery Validation:** Do not just break the system; verify that the system recovers without human intervention. If the system does not self-heal, the test has succeeded in identifying a requirement for an architectural improvement.

Hands-on Lab: Assessing Your Strategy

In this activity, you will identify the difference between reactive and proactive indicators for a specific service.

1. **Identify a failure mode:** Select a core service (e.g., an API gateway or a database).
2. **Define Reactive Indicators:** List the current alerts that fire when this service fails (e.g., "Pod CrashLoopBackOff" or "HTTP 5xx Errors").
3. **Define Proactive Experiments:** Design a Krkn scenario that mimics the failure mode defined in step 1.
 - **Example:** If the service fails due to high memory, use a Krkn memory-hog scenario to see if the Cluster Autoscaler triggers **before** the application reaches a crash state.
4. **Evaluate:** Run the Krkn experiment and compare the outcome against your reactive alert logs. Did the system survive? Did the alerts fire correctly? Use the findings to tune your thresholds or update your recovery SOPs.

Evaluating system performance under disruption

Evaluating system performance under disruption

In the realm of Chaos Engineering, injecting a failure is only half the battle. The true value lies in measuring how your system behaves—and, more importantly, how it recovers—while under the stress of that disruption. Evaluating performance under disruption requires a shift from viewing systems as binary (up/down) to observing them as dynamic, evolving environments.

Defining the Steady State

Before any disruption, you must establish a **Steady State**. The Steady State represents the baseline performance of your system during normal operations. Without this baseline, you cannot quantify the impact of a disruption.

To define your Steady State, focus on the Golden Signals: * **Latency**: The time it takes for a request to be serviced. * **Traffic**: A measure of how much demand is being placed on your system (e.g., requests per second). * **Errors**: The rate of requests that fail, either explicitly (HTTP 500s) or implicitly (wrong content). * **Saturation**: How "full" your service is (e.g., CPU/Memory utilization).

Observability as a Foundation

You cannot evaluate what you cannot see. When Krkn executes a disruption, your observability stack (typically Prometheus and Grafana) must be configured to provide real-time feedback.

When evaluating performance under disruption, monitor: 1. **Metric Divergence**: How far do current latency and error rate metrics drift from the established baseline? 2. **Resource Contention**: Does the disruption of one component (e.g., pod churn) cause resource exhaustion in another (e.g., node pressure)? 3. **Alerting Efficacy**: Do your existing Prometheus alerts fire accurately during the event? If the system is suffering but no alerts trigger, your monitoring is as much a failure point as the software itself.

Analyzing Impact: Beyond Binary States

When the system is disrupted, evaluate the performance using these criteria:

- **Recovery Validation**: How long does it take for the system to return to its steady state once the disruption is removed (or while it is active, if self-healing is in place)?
- **Graceful Degradation**: Does the system fail partially? For example, in a microservices architecture, does a backend failure cause the entire frontend to white-screen, or does it return a cached response or an informative error message?
- **Cascading Failures**: Use observability to determine if a targeted disruption in one area causes a "domino effect" across unrelated services.

Hands-on Lab: Measuring Latency during Pod Disruption

In this lab, you will monitor API server latency during a simulated pod disruption scenario using Krkn.

Prerequisites

- A target cluster with **krkn-hub** installed.
- **prometheus** and **grafana** access to monitor metrics.

Steps

1. **Baseline Collection**: Observe your API server latency in Grafana for a 10-minute window of normal traffic. Document the P99 latency.

PromQL Example:

```
`histogram_quantile(0.99, sum(rate(apiserver_request_duration_seconds_bucket[5m]))  
by (le))`
```

2. **Execute the Disruption:** Using the Krkn CLI, initiate a pod-kill scenario targeting your application namespace. + [source,bash] ---- krkn run pod-disruption -c config.yaml ----
3. **Observe Performance Shifts:** Monitor the Grafana dashboard during the experiment. Watch specifically for:
 - Spikes in latency (P99).
 - Increases in 5xx error codes.
 - ReplicaSet reconciliation times (how quickly does Kubernetes spawn new pods?).
4. **Evaluate:** Once the experiment completes, compare the metrics collected during the disruption to your baseline.
 - **Question:** Did the latency remain within your defined SLO (Service Level Objective) threshold despite the pod loss?
 - **Action:** If the latency exceeded thresholds, identify the bottleneck—was it network overhead, database connection pooling, or cold-start times of the new pods?
5. **Clean up:** Ensure all disrupted pods have returned to a **Running** state and that metrics have normalized back to the steady-state baseline.

Implementing recovery validation and observability

Implementing recovery validation and observability

In Chaos Engineering, the value of an experiment is not determined by the disruption itself, but by the system's ability to maintain or recover its steady state. Implementing robust recovery validation and observability is critical to ensuring that failures do not cascade into catastrophic outages.

The Role of Recovery Validation

Recovery validation is the process of confirming that the system returns to a healthy state after a disruption is injected. Krkn handles this through two primary layers:

1. **Scenario-Specific Checks:** Most Krkn scenarios include built-in logic to verify if the targeted component recovers within a predefined duration.
2. **Holistic Health Signals (Cerberus):** Because a failure in one component often impacts dependent services, verifying the target alone is insufficient. We utilize **Cerberus** to monitor the health of the entire cluster.



If Cerberus is enabled, it acts as a global "kill switch." It continuously tracks the health signal of the cluster. If the health signal drops below acceptable thresholds during an experiment, Cerberus sends a signal to halt the chaos, preventing

further damage and protecting the environment.

Observability and Metrics Evaluation

To determine whether an experiment is a success or a failure, we evaluate the system against three pillars:

- **Component Recovery:** Did the specific target (e.g., a Deployment or StatefulSet) return to its expected replica count and running state within the defined time limit?
- **Cluster Health:** Using Cerberus, did the broader cluster metrics (API server latency, node health, etc.) remain within acceptable bounds?
- **In-Cluster Prometheus:** We use PromQL to query metrics such as throughput, latency, and error rates to confirm that user-facing traffic is successfully processed post-disruption.

Hands-on Lab: Configuring Health Signals

In this exercise, you will learn how to integrate recovery validation into your workflow.

Step 1: Define the Steady State

Before starting, ensure you have a baseline for your application using Prometheus:

```
# Example: Monitor request success rate for your service
sum(rate(http_requests_total{status=~"2.."}[5m])) / sum(rate(http_requests_total[5m]))
```

Step 2: Enable Cerberus for Global Monitoring

To protect your environment, configure your Krkn execution to interface with Cerberus. Add the following to your configuration file:

```
cerberus:
  enabled: true
  url: "http://cerberus-service.cerberus.svc.cluster.local"
  check_period: 30
```

Step 3: Execute and Observe

1. Run your chosen Krkn scenario.
2. During execution, observe the Krkn logs. You will see output indicating that the controller is watching for recovery.
3. Simulate a scenario where the application fails to recover. Verify that Cerberus detects the degradation and halts the experiment.

Step 4: Validate Results

Post-experiment, verify the logs to see the final "Pass/Fail" status. A pass requires: * The target component reached the steady state (e.g., all replicas Ready). * Cerberus did not trigger a fail signal.

* Prometheus metrics returned to baseline levels within the defined timeframe.

Best Practices for Resilience

- **Minimal Initialization:** Design components for fast recovery logic. Long initialization times during restarts increase the risk of cascading failures.
- **High Availability (HA):** Ensure your application uses `PodDisruptionBudgets` and multi-zone deployments so that the recovery validation process doesn't wait on infrastructure that cannot recover due to lack of resources.
- **Threshold Tuning:** Set your failure detection thresholds based on real-world SLOs rather than aggressive, arbitrary timers, to avoid "false positive" failures in your chaos experiments.

Managing blast radius and rollback planning

Managing blast radius and rollback planning

In Chaos Engineering, the goal is to learn about system weaknesses without causing unintended downtime or service degradation. Managing the **blast radius** (the scope of the impact) and establishing robust **rollback planning** are the two primary safety pillars of any Krkn experiment.

Defining the Blast Radius

The blast radius defines the boundaries of your experiment. When executing scenarios with Krkn, you must ensure that the disruption is constrained to the intended targets.

- **Target Selection:** Use Krkn's multi-cluster capabilities to precisely define which clusters or namespaces are subject to testing. By leveraging Open Cluster Management (OCM) or manual registration, you can isolate experiments to non-critical development or staging environments before promoting them to production.
- **Resource Scoping:** Within a cluster, limit the blast radius by targeting specific labels or deployments. Avoid running global disruptions (like killing all pods in a namespace) until you have verified the behavior on a subset of instances.
- **Sequential Execution:** Instead of testing the entire architecture at once, use sequential execution to validate one microservice or node group at a time. This allows you to observe the recovery of one component before triggering a dependency failure.

Rollback Planning and Safety Mechanisms

A rollback plan ensures that if a system does not recover as expected, the environment can be returned to a "steady state" immediately.

1. Define the Steady State

Before any experiment begins, you must explicitly define what "normal" looks like. Use PromQL to monitor:

- * **Latency:** The average response time of your services.
- * **Throughput:** The request count per second.
- * **Error Rate:** The percentage of HTTP 5xx responses.

2. Implementation of Stop Conditions

Krkn experiments should be configured with automated stop conditions. If your steady state metrics cross a pre-defined threshold during a scenario (e.g., error rates exceed 5%), the experiment must be terminated.

- **Observability Integration:** Link your Krkn scenarios to your monitoring stack (e.g., Prometheus/Grafana).
- **Automated Rollback:** Ensure that your experiment configuration includes **kill** signals or cleanup hooks that restore the desired replica count or network policy configuration if the scenario triggers an alert.

3. Manual Intervention (The "Big Red Button")

Regardless of automation, always have a manual intervention plan: * **Emergency Termination:** Know how to identify and delete the running Krkn job if the experiment spirals out of control. * **Human-in-the-loop:** Especially in production environments, ensure a subject matter expert is monitoring the dashboard throughout the experiment duration to manually trigger a rollback if necessary.

Hands-on Activity: Configuring a Controlled Experiment

In this activity, you will learn how to limit the blast radius for a pod-disruption scenario.



Ensure you are operating in a non-production cluster before proceeding.

1. **Step 1: Scope the target** Create a YAML configuration for a **pod-disruption** scenario. Limit the scope using the **selector** field to target only a specific application label rather than the entire namespace.

```
scenario: pod-disruption
target_pods:
  selector: "app=frontend"
  namespace: "testing-ns"
  count: 1 # Limit to only 1 pod to keep impact low
```

2. **Step 2: Set the Experiment Duration** Keep the duration short for your first run to minimize the blast radius.

```
duration: 60s # The experiment will automatically cease after 60 seconds
```

3. **Step 3: Define the Stop Condition** Configure a validation check that runs alongside the experiment. If the **frontend** service latency exceeds 500ms, the experiment must cease.

```
stop_conditions:
  - metric: "http_request_duration_seconds"
    threshold: 0.5
```

```
comparison: ">"
```

4. **Step 4: Execute and Monitor** Run the scenario using `krkctl`. Keep a terminal open to monitor the logs. If the system fails to recover within 30 seconds of the scenario finishing, manually revert the target deployment:

```
# Manual rollback command if automated recovery fails
kubectl scale deployment/frontend --replicas=3 -n testing-ns
```

5. **Step 5: Review Results** Use the Krkn output logs to compare the "steady state" against the "disrupted state." If the blast radius was too large (e.g., more services were impacted than intended), refine your selectors and re-run.

Exploratory manual testing and pipeline automation

Exploratory manual testing and pipeline automation

Chaos engineering is not a one-size-fits-all discipline. To build a truly resilient system, you must balance the agility of **exploratory manual testing** with the consistency of **pipeline automation**.

Exploratory Manual Testing

Exploratory testing is an ad-hoc, curiosity-driven approach. It is invaluable during the initial phases of service design or when investigating an obscure "heisenbug" that does not appear in automated logs.

- **When to use:** Use manual testing when you are first onboarding a service to Krkn, or when you want to "poke" a system to see how it reacts to unforeseen variables.
- **The Process:**
 1. **Identify a Hypothesis:** "If I kill the primary database pod, will the secondary promote within 30 seconds?"
 2. **Execute via Krkn:** Use the Krkn CLI or UI to trigger a targeted disruption.
 3. **Observe:** Monitor dashboard metrics in real-time.
 4. **Analyze:** Note the "blast radius"—did the failure cascade to unrelated services? Did the system self-heal?

Manual testing provides the intuition required to define the boundaries for what should eventually become a recurring automated test.

Pipeline Automation

While manual testing builds your understanding, **pipeline automation** provides the continuous validation necessary for high-velocity CI/CD environments. Integrating chaos experiments into your CI/CD pipeline ensures that new code deployments do not introduce regressions in system resiliency.

- **Standardizing Execution:** Automate the execution of Krkn scenarios as a gated step in your deployment pipeline.
- **Quality Gates:** Use the output of your chaos experiments (e.g., success/failure of recovery validation) to determine if a deployment should proceed. If a service fails to recover from a pod disruption within defined thresholds, the pipeline automatically halts the promotion to the next environment.
- **Consistency:** Automation removes the "human error" factor. By running the same disruption scenarios against every build, you establish a performance baseline over time.

Comparing Approaches

Feature	Exploratory Manual Testing	Pipeline Automation	Purpose
Discovery and learning	Verification and regression	Ad-hoc	Triggered by code changes/Schedule
Effort	High (human-led)	Low (machine-led)	Outcome
	Standardized resilience metrics		

Hands-on Lab: From Manual to Automated

In this exercise, we will manually test a pod disruption and then define an automated sequence.

Step 1: Exploratory Manual Execution

1. Navigate to your target cluster via the Krkn UI.
2. Select a `pod-disruption` scenario.
3. Set the `chaos_duration` to 60 seconds.
4. Execute the job and observe the logs.
5. Document the system behavior: How long did it take for the service to restore availability?

Step 2: Automating the Test

Once you have validated the manual scenario, transition it to a workflow: 1. Open **Chaos Studio**. 2. Drag and drop the validated `pod-disruption` node onto the canvas. 3. Add a `recovery-validation` node. 4. Connect them with a dependency arrow (ensuring the validation runs immediately after the disruption). 5. Save this as a reusable workflow model. 6. Trigger this workflow using a CLI call within your CI/CD runner (e.g., Jenkins or GitHub Actions): `krkn run --workflow-id <id> --pipeline -mode`



Always start with exploratory testing to define your "steady state." Once you understand how your system reacts, wrap those specific parameters into an automated workflow to prevent future performance regressions.

Turn risk into a testable experiment

Turn risk into a testable experiment

Transforming system risks into actionable chaos experiments requires a structured approach to validate service resilience:

- **Identify Failure Modes:** Pinpoint potential service risks and failure points within the infrastructure.
- **Define Steady State:** Establish baseline metrics for availability, latency, and throughput to characterize normal operations.
- **Formulate Hypotheses:** Create testable assumptions regarding how the system should behave when specific disruptions occur.
- **Scope the Experiment:** Determine the target environment, duration, and blast radius to minimize unnecessary impact.
- **Map to Krkn Scenarios:** Align identified risks with specific Krkn disruption scenarios to trigger the desired failure mode.
- **Establish Safety Controls:** Define clear monitoring criteria, stop conditions, and rollback procedures to maintain system stability during testing.

Identifying service risks and failure modes

Identifying service risks and failure modes

To build a resilient architecture, you must shift from a reactive "fix-it-when-it-breaks" mentality to a proactive strategy that anticipates failures. Identifying service risks and failure modes is the foundational step of the Chaos Engineering lifecycle. By mapping how components interact and where they are fragile, you can transform abstract risks into measurable, testable experiments.

The Anatomy of Failure

Before injecting chaos, you must deconstruct your services to understand their dependencies. Failure modes are not always catastrophic outages; they often manifest as subtle degradations.

Common failure modes to consider: * **Resource Exhaustion:** Memory leaks, CPU throttling, or storage disk saturation. * **Dependency Failure:** Latency spikes or unavailability of upstream/downstream APIs. * **Networking Issues:** Packet loss, DNS resolution failures, or partitioned subnets. * **Configuration Drift:** Incorrect environment variables or outdated security policies leading to unauthorized access or crashes.

Mapping Risks to Chaos Scenarios

Once you identify a potential failure mode, you must determine how to replicate it using Krkn. Krkn provides specific disruption scenarios that allow you to simulate these conditions safely.

Identified Risk	Failure Mode	Krkn Scenario Category
Database connection loss	Network partition/Latency	Network Chaos
Unhealthy Node	Memory pressure	Node/Pod Disruption
Service Bottleneck	CPU contention	Resource Chaos
Expiring Certs	Security/Communication failure	Configuration Chaos

Assessing System Fragility

Use the following methodology to categorize and prioritize your risks:

1. **Define Steady State:** What does "normal" look like? Document your baseline Key Performance Indicators (KPIs) such as response latency (p99), request throughput, and error rates.
2. **Analyze Dependencies:** Map the service graph. If Service A depends on Service B, and Service B experiences a 500ms latency increase, how does that impact the user-facing latency of Service A?
3. **Identify Blast Radius:** Determine the scope of the potential failure. Does this affect a single pod, an entire availability zone, or a global multi-cluster deployment?
4. **Evaluate Observability Gaps:** If you cannot monitor the failure, you cannot test it. Ensure your Prometheus/Grafana dashboards are capable of alerting on the specific metrics tied to this failure mode.

Hands-On Activity: Identifying Failure Modes

In this activity, you will conduct a "Failure Mode and Effects Analysis" (FMEA) for a sample application to prepare for your first Krkn experiment.

Prerequisites

- Access to an OpenShift cluster.
- A deployed application to serve as the target.
- Access to **oc** CLI.

Steps

1. **Identify the critical path:** List the components that must remain functional for the user to complete a core task (e.g., "Add to Cart").
2. **Brainstorm disruptions:** For each component in the critical path, ask: "What happens if this container is killed?" or "What happens if this API becomes unresponsive?"
3. **Draft the Experiment:**
 - **Goal:** Test the application's ability to handle pod loss.

- **Hypothesis:** If a pod is terminated during a traffic spike, the horizontal pod autoscaler (HPA) will spin up a replacement, and the p99 latency will not increase by more than 10%.
 - **Stop Condition:** If error rates exceed 5% during the test, terminate the experiment immediately.
4. **Prepare for Krkn:** Use this hypothesis to select the appropriate Krkn scenario (e.g., **pod-reaper**) in the next phase of your workflow.



Always start small. Do not test in production until you have validated your failure modes and recovery procedures in a non-production staging environment. Use **Machine Health Checks** in your cluster to ensure that the infrastructure can remediate unhealthy nodes automatically before you attempt to force-trigger these failures manually.

Defining steady state and test hypotheses

Defining steady state and test hypotheses

In Chaos Engineering, the "steady state" is your baseline for normal operations. Without a clearly defined steady state, it is impossible to determine whether an injected failure caused a meaningful service degradation or if the system was already underperforming.

Understanding Steady State

The steady state represents the expected behavior of your system under normal operating conditions. It is not defined by the absence of errors, but by the performance boundaries within which your system fulfills its business requirements.

To define your steady state, focus on these key pillars: * **Availability:** Is the service reachable? (e.g., uptime, pod health). * **Latency:** How long does it take for a request to be fulfilled? (e.g., P99 response time). * **Throughput:** How much traffic is the system processing? (e.g., requests per second). * **Error Rates:** What is the ratio of successful to failed requests? (e.g., HTTP 5xx codes).

Defining Metrics for Evaluation

Before running any Krkn experiment, you must establish the metrics that represent your steady state. Use PromQL to define these in your monitoring system (such as Prometheus or Thanos).

Example: Defining an API Latency Baseline

```
# Average request duration over the last 5 minutes
rate(http_request_duration_seconds_sum[5m]) /
rate(http_request_duration_seconds_count[5m])
```

Formulating a Test Hypothesis

A test hypothesis is a structured prediction of how your system will react to a specific disruption. It follows a scientific approach: "If we perform **[Action]**, then **[System]** will **[Expected Result]**."

The Hypothesis Framework

When using Krkn to inject chaos, structure your hypothesis around the **Blast Radius** and **Recovery Validation**:

1. **Observation:** "Our API service currently maintains a P99 latency of < 200ms."
2. **Disruption:** "If we terminate 50% of the API pods in one availability zone..."
3. **Expected Outcome:** "...the system will automatically reschedule pods, and the P99 latency will not exceed 500ms for more than 60 seconds."
4. **Stop Condition:** "If the error rate exceeds 5% or the latency remains high for > 3 minutes, the experiment must abort and rollback."

Hands-on Activity: Defining Your Experiment Baseline

Follow these steps to establish your steady state before triggering a Krkn experiment.

Step 1: Document the "Golden Signals" Choose one service you intend to test. Record the following values during a period of normal traffic: * **Current P99 Latency:** `ms` * **Current Request Throughput:** `req/s` * **Current Error Rate:** `_ %`

Step 2: Define the Hypothesis in your Krkn Configuration When configuring your Krkn scenario, ensure your YAML includes clear criteria for success.

```
# Example structure for hypothesis definition
chaos_experiment:
  name: api-pod-disruption
  hypothesis: "System should recover within 60s"
  steady_state_metrics:
    - name: api-latency
      query: "histogram_quantile(0.99,
sum(rate(http_request_duration_seconds_bucket[5m])) by (le))"
      threshold: 0.5 # 500ms limit
  stop_conditions:
    - error_rate > 0.05
```

Step 3: Establish the "Blast Radius" Identify which specific services, namespaces, or nodes will be affected. Limiting the blast radius allows you to test hypotheses in a controlled manner without impacting the entire production environment.

Verification: After running your experiment, compare the recorded metrics from Step 1 against the metrics gathered during the disruption. If the deviation is within your established threshold, your system is considered resilient to this specific failure mode.

Setting target, duration, and blast radius

Setting target, duration, and blast radius

In Chaos Engineering, the precision of your experiment design directly impacts the reliability of your findings. When using Krkn, defining the boundaries of an experiment ensures that you can test failure modes without causing catastrophic, uncontrolled outages.

Identifying Experiment Boundaries

Before executing a scenario, you must define the "scope" of the impact. This involves three primary variables:

1. Target

The **target** defines the infrastructure or application components that will be subject to the disruption. * **Granularity:** You can target specific namespaces, labels, or entire clusters. * **Cluster Discovery:** Krkn integrates with ACM/OCM to discover available infrastructure. When configuring your experiment, use these providers to select the exact clusters or worker nodes where the failure will occur. * **Best Practice:** Start with a small, non-critical subset of your environment to validate that the scenario behaves as expected.

2. Duration

The **duration** defines how long the disruption will persist. * **Short Bursts:** Useful for testing how systems handle transient network spikes or momentary pod instability. * **Extended Windows:** Used to verify that recovery controllers (like the Kubelet or Kubernetes Deployment controllers) effectively reconcile the system back to the desired steady state over time. * **Calculation:** Ensure your duration is long enough to exceed the typical time-to-detect (TTD) and time-to-recover (TTR) metrics of the system under test.

3. Blast Radius

The **blast radius** defines the "reach" of the experiment. It is the percentage or specific number of resources affected. * **Controlled Impact:** If you are testing a service with 10 replicas, a blast radius of 10% (1 replica) allows you to observe how the load balancer handles a single pod failure. * **Scaling:** As confidence grows, you can increase the blast radius to test the limits of your system's redundancy—for example, by killing 50% of pods or inducing latency across entire availability zones. * **Risk Mitigation:** Always define a **Stop Condition**—a threshold that, if crossed, will automatically terminate the experiment to prevent the blast radius from expanding beyond your design.

Hands-on Lab: Configuring Experiment Parameters

In this lab, you will configure a pod-disruption scenario using the Krkn wizard.

Prerequisites

- Access to a Krkn instance with at least one target cluster registered.
- Appropriate user permissions to execute chaos jobs.

Step-by-Step Configuration

1. **Initiate the Scenario:** Navigate to the **Run Scenarios** dashboard in your Krkn console. Select the **pod-disruption** scenario.
2. **Define Target Clusters:** Using the **Provider Configuration** section, select your target cluster(s). Note that selecting multiple clusters will execute the scenario across all selected targets simultaneously.
3. **Set Parameters:** In the configuration wizard, input the following:
 - **Target Namespace:** **my-app-production**
 - **Label Selector:** **app=frontend**
 - **Duration:** **300s** (5 minutes)
 - **Blast Radius:** Set the **expected_impact** to **20%** of the targeted pods.
4. **Validation:** Review the summary panel. Ensure the target cluster IDs match your intended test environment and the blast radius parameters align with your defined SLOs (e.g., ensuring you don't exceed the total replica capacity).
5. **Execute:** Click **Run**. Observe the job status in the logs. Monitor your observability platform (e.g., Prometheus/Grafana) to ensure that the blast radius remains contained within the specified parameters.



If the disruption causes an unexpected drop in throughput beyond your defined metrics, use the **Stop Experiment** function immediately to evaluate if your blast radius was set too aggressively for the current system load.

Establishing metrics for availability, latency, and throughput

Establishing metrics for availability, latency, and throughput

In the context of Chaos Engineering with Krkn, moving from intuition-based testing to evidence-based validation requires a rigorous approach to metrics. To determine if your system has maintained its "steady state" during a disruption, you must define and monitor clear metrics for Availability, Latency, and Throughput.

Understanding the Golden Signals

When identifying service risks, we align our hypotheses with the "Four Golden Signals" of monitoring. For Krkn experiments, we prioritize the following three:

- **Availability:** The percentage of requests that succeed (e.g., HTTP 2xx/3xx codes vs. 5xx). This directly maps to your Service Level Objectives (SLOs).
- **Latency:** The time it takes to service a request. It is critical to distinguish between latency for successful requests and latency for failed requests.
- **Throughput:** The amount of work the system is performing, typically measured in requests per second (RPS) or transactions per second (TPS).

Defining Steady State Metrics

Before executing a Krkn scenario, establish a baseline for your metrics. If you do not know what "normal" looks like, you cannot mathematically prove that a disruption caused an anomaly.

Best Practices for Metric Selection

- **Use Quantiles for Latency:** Avoid using averages. An average can mask the experience of the "long tail" of users. Use p95 or p99 latency to capture the experience of users impacted by the disruption.
- **Define Success Thresholds:** Establish the limit of acceptable degradation. For example: "During a pod-kill experiment, p99 latency must not exceed 500ms for more than 30 seconds."
- **Contextualize Throughput:** Ensure throughput metrics account for background noise. If your experiment targets a specific microservice, isolate its throughput metrics from the global application traffic.

Mapping Metrics to Krkn Experiments

When planning your Krkn scenario, you must define the "Observation Window." This is the time during which you collect metrics to validate your hypothesis.

```
# Example structure of a Krkn experiment configuration
experiment:
  name: "pod-disruption-latency-test"
  metrics:
    - name: "http_requests_total"
      query: "sum(rate(http_requests_total{status=~'2..'}[1m]))"
    - name: "p99_latency"
      query: "histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket[1m])))"
  steady_state_thresholds:
    max_latency_increase: "20%"
    min_availability: "99.9%"
```

Hands-on Activity: Establishing a Monitoring Baseline

In this exercise, you will define the metrics needed to validate the impact of a pod disruption.

Prerequisites: * A running application with an exposed Prometheus metrics endpoint. * Access to a Grafana dashboard or PromQL console.

Step 1: Baseline Recording 1. Observe your application in a steady state for 10 minutes. 2. Capture the current **p99** latency and **error_rate** (5xx codes). 3. **Record these values.** These are your "Steady State" constants.

Step 2: Defining the Stop Condition Use your baseline to define a stop condition in your Krkn config. If these thresholds are breached during a disruption, the operator must stop the experiment to prevent a total outage. * **Availability Threshold:** If **error_rate** > 5% for more than 1 minute, trigger rollback. * **Latency Threshold:** If **p99_latency** > baseline * 1.5, trigger rollback.

Step 3: Verification Once you run the Krkn experiment, compare the post-experiment report against these recorded baselines. If the availability dropped but stayed within your defined "acceptable limit," the service is considered resilient to that specific failure mode.



Always ensure that your metric scraping interval is shorter than the duration of the chaos experiment. If your experiment runs for 60 seconds but your Prometheus scrape interval is 60 seconds, you may miss the spike in latency or throughput changes.

Planning monitoring, stop conditions, and rollbacks

Planning Monitoring, Stop Conditions, and Rollbacks

In Chaos Engineering, the goal is not to break systems for the sake of destruction, but to learn how systems respond to failure. Without robust monitoring, automated safety triggers (stop conditions), and clear rollback plans, an experiment shifts from a controlled test to an uncontrolled incident.

Monitoring for Steady State

Before initiating any disruption with Krkn, you must establish a baseline. This "steady state" is defined by specific observability metrics that indicate the application is functioning correctly under normal load.

- **Availability:** Success rates of HTTP requests or gRPC calls.
- **Latency:** P99 response times for critical transactions.
- **Throughput:** Requests per second (RPS) handled by the service.
- **Resource Health:** CPU, Memory, and Disk utilization of the targeted nodes/pods.

Using Krkn, these metrics should be tracked via a monitoring stack (such as Prometheus). You will use these metrics not only to observe the impact of the chaos but as inputs for your **Resiliency Score**.

Defining Stop Conditions

Stop conditions act as the "emergency brake" for your experiment. If your metrics deviate beyond a predefined threshold, the experiment must terminate immediately to protect the production environment and prevent a full-scale outage.

When mapping risks to Krkn scenarios, define stop conditions based on: * **SLO Breaches:** If the error rate exceeds 5% for more than 60 seconds. * **Dependency Failure:** If a secondary service (e.g., a database or message broker) becomes unreachable. * **Blast Radius Expansion:** If the disruption spreads beyond the intended target namespace or cluster nodes.



In the Krkn Operator, ensure that your monitoring alerts are configured to notify the SRE team, but also consider automating the "kill switch" by triggering a rollback if your critical SLOs are violated during the execution of a scenario.

Rollback Planning

Rollback planning is the process of defining the steps required to return the system to its steady state once the disruption duration has elapsed or if a stop condition is triggered.

1. **Automated Recovery:** Most Krkn scenarios (such as Node Shutdown or Resource Hogs) include built-in recovery logic. For example, the **Power Outage** scenario automatically triggers a restart of nodes after the specified duration.
2. **Manual Intervention:** If an experiment causes an unexpected state (e.g., a process fails to restart after a CPU Hog), have a clear "Runbook" ready to manually restore service.
3. **Validation:** After a rollback is triggered, you must verify that:
 - Pods are in a **Running** or **Ready** state.
 - Traffic is flowing through all nodes.
 - Logs show no residual errors caused by the aborted experiment.

Hands-on Lab: Configuring Stop Conditions in Chaos Studio

In this lab, you will configure a Chaos Studio workflow to monitor an application and define a stop condition based on latency.

1. **Step 1: Set the Hypothesis** Identify a microservice and define its target P99 latency. For this lab, we will use a threshold of 500ms.
2. **Step 2: Configure PromQL Monitoring** Within your Chaos Studio workflow node, add a "Resiliency Score" configuration.
 - Use the following PromQL to track latency:

```
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

3. **Step 3: Define the Stop Condition** Configure the workflow node to "Abort on Failure." Map this to the PromQL query above. If the resulting value exceeds **0.5** (500ms), the Krkn Operator will stop the disruption scenario.
4. **Step 4: Execute and Observe** Run your workflow. Observe the **Jobs** page in the Krkn dashboard. If the application latency spikes due to your chaos injection, verify that the scenario terminates early and that the system returns to the baseline metrics within your expected recovery time.
5. **Step 5: Verify Recovery** Check the logs in the Krkn dashboard to ensure that the recovery phase of the scenario completed successfully and the cluster status returned to **Ready**.

Mapping risks to Krkn disruption scenarios

Mapping Risks to Krkn Disruption Scenarios

In Chaos Engineering, the transition from identifying a theoretical failure mode to executing a

controlled experiment is where resilience is truly built. Mapping your system's risks to specific Krkn disruption scenarios ensures that your testing is intentional, measurable, and safe.

Understanding the Risk-to-Scenario Mapping

Risk mapping involves identifying a component's "failure mode" and selecting the appropriate Krkn scenario that mimics the real-world conditions causing that failure.

When mapping, categorize your risks into two primary layers: * **Infrastructure-Level Risks:** Node failures, network partitioning, storage latency, or cloud-provider API throttles. * **Application/Workload-Level Risks:** Pod crashes, container resource exhaustion (CPU/Memory), dependency timeouts, or service mesh traffic failures.

Identifying Scenarios via Krkn-Hub

The link between your risk and the execution is the **Krkn-hub**. Krkn provides specialized container images, each designed to inject a specific type of fault. When you identify a risk—for example, "What happens if our database service becomes unreachable due to a network partition?"—you map it to the corresponding network latency or packet loss scenario provided by the Krkn ecosystem.

Mapping Framework Table

Identified Risk	Failure Mode Category	Recommended Krkn Scenario Type	:---	:---	:---	
Database connection timeout	Network	Network Latency / Packet Loss	High pod churn under load	Resource	Pod Disruptions / Memory Stress	Worker node failure
Infrastructure	Node Shutdown / Node Power-off	Dependent service latency	API / Communication	Service Mesh /	Proxy Faults	

Hands-on Activity: Defining the Experiment Mapping

In this activity, you will map a high-level risk to a specific Krkn execution plan.

Prerequisites

- Access to a Kubernetes cluster.
- `krknctl` installed and configured.
- An identified service with a clear "steady state" (e.g., a web service returning 200 OK).

Step 1: Define the Steady State

Before mapping the risk, define your baseline. * **Metric:** HTTP 200 responses per second. * **Threshold:** > 95% success rate over a 1-minute window.

Step 2: Select the Krkn Disruption Scenario

Assume your risk is: "The application will fail if the front-end pod is killed suddenly by the scheduler." 1. Navigate to the `krkn-hub` documentation. 2. Locate the **Pod Disruption** scenarios. 3.

Choose the **pod-delete** scenario, which forces the Kubernetes scheduler to terminate and restart specific pods, simulating unexpected preemption.

Step 3: Configure the Risk Mapping

Create a configuration file (YAML) that maps the risk to the execution parameters:

```
# Example mapping configuration
experiment:
  scenario: "pod-delete"
  target_cluster: "production-cluster-01"
  blast_radius:
    label_selector: "app=frontend"
    count: 1 # Only terminate one pod at a time to control blast radius
    duration: 300s # The window to observe the impact on the steady state
```

Step 4: Validate the Mapping

Execute the scenario using **krknctl** to see if the system recovers as expected:

```
krknctl run pod-delete --config experiment-map.yaml
```

Best Practices for Mapping

- **Start Small:** Map risks to scenarios that affect only a single node or a single pod initially.
- **Validate Metrics:** Ensure that your metrics (Prometheus/Grafana) are actively scraping during the scenario execution. If you cannot measure the impact, the risk mapping is incomplete.
- **Stop Conditions:** Always map a "Stop Condition" to your risk. If the error rate exceeds your defined threshold (e.g., > 10% 5xx errors), the Krkn Operator should automatically roll back the experiment to preserve system integrity.

Krkn Operator fundamentals

Krkn Operator fundamentals

The Krkn Operator serves as the central control plane for managing resilience experiments across Kubernetes clusters. It abstracts the complexity of orchestrating disruptions by providing a unified interface for scenario management and execution.

Key operational capabilities include:

- **Cluster Management:** Facilitates discovery and target management using integration with ACM and OCM.
- **Scenario Execution:** Provides access to scenario registries to deploy and manage chaos jobs with defined parameters.
- **Observability:** Enables monitoring of experiment logs and status reports to track real-time failure progression.
- **Resiliency Analysis:** Integrates with Chaos Studio to visualize experiment outcomes and calculate Resiliency Scores.
- **Multi-cluster Orchestration:** Supports the configuration and execution of complex, multi-cluster disruption scenarios from a central control point.

Managing target clusters and user permissions

Managing target clusters and user permissions

In the Krkn ecosystem, security and operational safety are foundational. Because chaos experiments intentionally induce failures, access control is not merely a feature—it is a critical safeguard. Krkn utilizes a group-based permission model to ensure that only authorized users can impact specific environments.

The Permission Model

Krkn operates on a strict **Group-Based Access Control (GBAC)** architecture. Every user must be associated with at least one group; a user without a group assignment has no access to the platform.

Permissions are not assigned to individual users directly, but are inherited from the groups they belong to. This approach ensures consistent policy enforcement across teams.

Group Configuration

When an Administrator creates a group, they define the operational boundaries for all members of that group:

- **Cluster Scoping:** Defines which target clusters are visible and available for experimentation.
- **Permissions:** Grants specific actions, such as **View**, **Run**, or **Delete**.
- **Registries:** Defines which scenario registries are visible to the group, allowing teams to use

specific sets of pre-validated experiments.



If a user is not assigned to a group, they are effectively "locked out" of the Krkn platform, preventing accidental or unauthorized execution of disruptive experiments.

Administrative Roles and Responsibilities

Administrators manage the platform infrastructure, serving as the "Source of Truth" for security policies. Their responsibilities include:

- **Cluster Lifecycle:** Registering and removing target clusters from the Krkn platform.
- **User/Group Management:** Creating groups, defining the scope, and mapping users to those groups.
- **Provider Integration:** Configuring target providers, such as ACM (Advanced Cluster Management) integration, to automate cluster discovery.
- **Registry Management:** Setting up private registries and defining group-level visibility.

Operational Permissions

Once a user is assigned to a group, their interaction with Krkn is governed by the permissions granted by that group. These permissions determine the scope of a user's chaos engineering activities:

Permission	Description
<code>View</code>	Allows users to monitor the status of jobs and view experiment logs.
<code>Run</code>	Required to execute scenarios, use the cluster terminal, and design workflows in Chaos Studio.
<code>Delete</code>	Allows users to manage and remove past job records or artifacts.

Hands-on Lab: Configuring Access

This activity guides you through the process of setting up a secure testing environment for a new team.

Prerequisites

- Administrator access to the Krkn platform.
- A target cluster registered and ready for connection.

Step 1: Create a Restricted User Group

1. Log in as an **Administrator**.
2. Navigate to the **Access Control** or **Groups** management section.
3. Click **Create Group**.
4. Name the group (e.g., `chaos-testers-prod`).
5. Select the **Target Clusters** from the list that this group is permitted to disrupt.

6. Assign the **Run** and **View** permissions.

Step 2: Assign Users to the Group

1. In the same **Groups** management section, navigate to the **Users** tab.
2. Select the target user(s).
3. Assign them to the **chaos-testers-prod** group.

Step 3: Verify Permissions

1. Log out as the Administrator.
2. Log in as a user assigned to the **chaos-testers-prod** group.
3. Navigate to the **Target Clusters** dashboard.
4. **Verification:** Ensure you can only see the clusters defined in your group. Attempt to trigger a "No-Op" scenario to confirm you have **Run** permissions.



Always follow the **Principle of Least Privilege**. Only grant **Delete** permissions to users who are specifically responsible for cleaning up experiment artifacts to prevent accidental deletion of important audit logs.

Working with scenario registries

Working with scenario registries

In the Krkn ecosystem, **Scenario Registries** serve as the centralized repository for disruption logic. They act as a catalog of pre-defined chaos experiments that you can version, share, and deploy across your managed clusters. Understanding how to interact with these registries is fundamental to ensuring consistent and reproducible chaos engineering practices.

Understanding Scenario Registries

A scenario registry is essentially a collection of YAML-based configuration files that define the "how" and "what" of a chaos experiment. By utilizing registries, teams can move away from ad-hoc, manual script execution toward a standardized library of vetted failure modes.

Registries allow you to:

- * **Centralize:** Maintain a single source of truth for chaos scenarios.
- * **Version:** Track changes to experiments over time, ensuring auditability.
- * **Reuse:** Apply the same validated disruption across multiple development, staging, and production environments.

Anatomy of a Scenario Configuration

Each scenario within a registry is structured to provide the Krkn Operator with the necessary instructions to induce a specific failure. Configuration parameters are generally categorized into:

- **Mandatory:** Parameters required for the scenario to execute (e.g., target namespace, pod labels).
- **Optional:** Fine-grained controls that allow you to tune the behavior, such as specific timing

intervals or packet loss percentages in a Network Chaos scenario.

- **Global:** Framework-level settings that affect how the scenario interacts with external observability tools like Prometheus, Elasticsearch, or Cerberus.

File Management and Customization

Krkn provides a robust interface for managing these configurations. You can upload and manage text-based configuration files directly within the platform, making them immediately available for use in Chaos Studio workflows or direct job execution.



Always validate your configuration files against your organizational standards before committing them to a shared registry. Ensure that mandatory fields are correctly defined to prevent "silent failures" during experiment execution.

Hands-on Activity: Exploring and Configuring a Scenario

This lab activity guides you through the process of locating a scenario in the registry and preparing it for execution.

1. **Navigate to the Scenario Catalog** Log into the Krkn dashboard and locate the **Scenarios** tab. Browse the available library, noting categories like **Pod Network Scenario** or **Persistent Volume Fill**.
2. **Select and Inspect** Click on the **Pod Network Scenario** documentation link. Review the parameters required:
 - Identify the **Mandatory** fields (e.g., **label_selector**).
 - Identify the **Optional** fields (e.g., **duration**, **traffic_type**).
3. **Upload a Custom Configuration** If you have a specific requirement (e.g., a 60-second network latency injection), create a **network-latency.yaml** file: `[source,yaml] --- scenario: network-latency duration: 60s latency: 100ms target_label: app=my-service ---` Navigate to the **File Management** section and upload this file to the registry.
4. **Validate the Configuration** Once uploaded, select the scenario and use the configuration panel to verify that the Krkn operator correctly parses your YAML fields. Ensure all mandatory labels are properly mapped to your cluster's actual deployment labels.
5. **Prepare for Execution** Save your configuration as a preset. This enables you to reuse this exact failure mode in future workflows without re-entering the parameters manually.

Next Steps

After configuring your scenario, you are ready to transition to the **Run** phase. By triggering a job from your configured registry entry, you can monitor the real-time impact of the disruption through the **Jobs** dashboard and observe how your application's steady state is affected.

Executing jobs and managing logs

Executing jobs and managing logs

In the Krkn ecosystem, the lifecycle of a chaos experiment centers on the **Jobs** interface. This is where your hypotheses meet reality, allowing you to monitor real-time disruptions, inspect granular execution data, and troubleshoot unexpected outcomes.

Executing Chaos Experiments

When you launch a scenario, you transition from configuration to execution. The Krkn platform automates the orchestration of these experiments, managing the lifecycle of the disruption pods, target resource isolation, and data collection.

Launching a Job

To initiate an experiment: 1. Navigate to your desired scenario within the registry. 2. Configure the experiment parameters (blast radius, duration, and target namespace/labels). 3. Click **Run**. 4. You will be automatically redirected to the **Jobs** dashboard.

Monitoring Real-Time Status

The Jobs dashboard provides a high-level view of all scenario executions within your group. Key metrics displayed include: * **Status:** (Pending, Running, Succeeded, Failed, or Interrupted). * **Execution Timestamp:** Start and end times for duration tracking. * **Cluster Targets:** The specific infrastructure or application targets affected by the run.

Managing and Inspecting Logs

Once a job is running or completed, managing the output is critical for validating whether your service maintained its steady state.

Accessing Console Logs

To inspect the internal behavior of the Krkn operator during the execution: 1. Locate the specific job in the **Jobs** list. 2. Select the **Logs** tab associated with that job. 3. Review the output for: * **Disruption initiation:** Confirming that the targeted resources were successfully impacted. * **Recovery sequence:** Observing the automated cleanup or workload recovery steps. * **Errors/Warnings:** Identifying issues with permissions, connectivity, or invalid resource selectors.

Permissions and Visibility

The Krkn platform enforces a granular security model regarding job management: * **View Access:** Users with "View" permissions can inspect job outcomes, graph run nodes, and Resiliency Scores, but are restricted from executing or deleting experiments. * **Group Isolation:** By default, the dashboard displays only jobs associated with your specific group. * **Cluster-Wide Transparency:** A cluster-wide indicator shows the names of scenarios running across other groups, providing visibility without compromising the security of unauthorized jobs. * **Deletion:** Removing historical job data requires explicit "Delete" permissions to ensure audit trails remain intact.

Hands-on Activity: Observing a Job Lifecycle

In this activity, you will execute a pod-disruption experiment and verify the logs to confirm the

system's behavior.

1. Launch the Experiment:

- Select a standard pod-disruption scenario from your registry.
- Set a duration of 120 seconds.
- Click **Run**.

2. Monitor Execution:

- Observe the status change from **Pending** to **Running** on the **Jobs** page.
- Refresh the page to see the real-time status updates as the chaos progresses.

3. Inspect Logs:

- Once the status reaches **Succeeded**, click on the job ID.
- Navigate to the **Logs** tab.
- Search for the log entry **Disruption successfully initiated** and verify the recovery timestamp to see if the workload scaled back to its original steady state.

4. Analyze Results:

- Review the **Resiliency Score** generated post-execution. If the score is lower than expected, cross-reference the console logs with your application metrics to identify the specific moment of SLO degradation.

Utilizing ACM and OCM cluster discovery

Utilizing ACM and OCM Cluster Discovery

In large-scale Kubernetes environments, manually managing chaos experiment targets can become an administrative burden. The Krkn Operator simplifies this by integrating with **Open Cluster Management (OCM)** and **Red Hat Advanced Cluster Management (ACM)**, enabling automated discovery of managed clusters.

Understanding OCM/ACM Integration

The Krkn Operator does not strictly require ACM or OCM to function; manual cluster registration is fully supported and offers feature parity. However, integrating with OCM/ACM provides:

- **Automated Discovery:** Automatically populates your available targets without manual configuration.
- **Centralized Management:** Leverages your existing multi-cluster governance infrastructure.
- **Operational Efficiency:** Reduces the risk of human error when onboarding new clusters for chaos testing.



The integration with OCM/ACM is optional. It is designed to automate the discovery process, not to provide additional chaos functionality. All clusters—whether discovered via ACM/OCM or added manually—possess the same capabilities within

the Krkn Operator.

Enabling Cluster Discovery via Helm

To enable the integration during the installation of the Krkn Operator, you must set the `acm.enabled` flag to `true` in your Helm deployment.

```
helm install krkn-operator oci://quay.io/krkn-chaos/charts/krkn-operator \
  --version 0.4.5-beta \
  --namespace krkn-operator-system \
  --create-namespace \
  --set acm.enabled=true
```

Compatibility and Requirements

Before enabling integration, ensure your environment meets the tested compatibility requirements. Using unsupported versions may lead to discovery failures or inconsistent behavior.

Table 1. Verified OCM Compatibility Matrix

OCM Version	Status	
1.1.0	Tested	
1.2.0	Tested	
1.3.1	Tested	



If you are running an older version of ACM not listed in the table, the integration may still function; however, it is not actively tested or supported. We recommend upgrading to one of the validated versions above to ensure stability in your chaos experiments.

Hands-on Activity: Verifying Cluster Discovery

In this activity, you will verify that your clusters are correctly being identified by the Krkn Operator.

- Check Pod Status:** Ensure the `krkn-operator` is running in the `krkn-operator-system` namespace.
`kubectl get pods -n krkn-operator-system`
- Verify Configuration:** Inspect the operator deployment configuration to confirm that the OCM integration is active.
`kubectl get deployment krkn-operator-controller-manager -n krkn-operator-system -o yaml | grep -A 2 acm`
- Validate Discovery:** Use the Krkn Operator CLI or API to list the discovered clusters.
 - If using the Chaos Studio interface, navigate to the **Clusters** tab.
 - If using the CLI, run the command: `krkn-cli list-clusters`
- Troubleshooting:**
 - If clusters do not appear, verify that the `krkn-operator` has the necessary `ClusterRole`

permissions to read the OCM/ACM resources.

- Ensure the **ManagedCluster** custom resources in the OCM hub cluster are in the **Ready** state.

By successfully integrating OCM/ACM, you lay the foundation for automated, multi-cluster resilience testing, allowing you to scale your chaos engineering efforts across the entire fleet seamlessly.

Configuring multi-cluster scenario execution

Configuring multi-cluster scenario execution

In complex, distributed environments, testing resilience often requires verifying how a failure propagates across multiple infrastructure boundaries. Krkn's architecture enables you to execute chaos scenarios simultaneously across several clusters, providing a comprehensive view of how your global control plane and individual workloads handle synchronized disruption.

Understanding Multi-Cluster Execution

When you configure multi-cluster scenario execution in Krkn, you are not just running independent tests; you are orchestrating a coordinated failure experiment. By selecting multiple target clusters, Krkn triggers the scenario on each target environment concurrently, allowing you to observe cross-cluster latency, service availability, and global recovery patterns.

Prerequisites for Multi-Cluster Runs

Before configuring your execution, ensure:

- * **Connectivity:** The Krkn controller has the necessary **kubeconfig** or ServiceAccount tokens to access all target clusters.
- * **Permissions:** The Krkn operator must have appropriate RBAC permissions (cluster-admin or scoped permissions) on all target clusters to perform the requested disruption (e.g., pod kills, node pressure, network latency).
- * **Registry Access:** The scenario registries must be reachable from the target clusters to pull the necessary chaos logic.

Configuring Execution via the Guided Wizard

To ensure a standardized approach, Krkn provides a step-by-step wizard to configure and deploy scenarios across a fleet of clusters.

Step-by-Step Configuration

1. **Initiate the Run:** From the Krkn dashboard, navigate to the **Run Scenarios** section and select the **Run Scenarios Wizard**.
2. **Select Target Clusters:**
 - The interface will display a list of discovered clusters (utilizing ACM or OCM cluster discovery).
 - Check all clusters that should be subjected to the disruption.
 - **Note:** All selected clusters will execute the scenario simultaneously upon initiation.
3. **Select Scenario:** Choose the desired chaos scenario from your registered libraries.

4. Define Experiment Parameters:

- Specify the blast radius (e.g., number of pods, percentage of nodes).
- Set the duration of the impact.
- Define the steady-state metrics and stop conditions.

5. **Review and Execute:** Validate the cluster selection and parameters. Once triggered, Krkn initiates the pods on the selected target clusters to execute the experiment.

Monitoring and Analysis

Once the multi-cluster execution begins, you can track the experiment status globally:

- **Execution Outcome:** Monitor the status (Success/Failure) for each individual cluster.
- **Centralized Logs:** Access the console logs for the specific Krkn pods generated on each cluster. This is critical for identifying why a scenario might have succeeded on one cluster but failed on another (often due to environmental configuration drift).
- **Observability Correlation:** If your environment is integrated with a centralized monitoring tool (like Prometheus/Grafana), use the timestamps from the Krkn execution logs to correlate the disruption with metrics across all target clusters simultaneously.

Best Practices for Multi-Cluster Chaos



Manage the Blast Radius: Running a high-impact scenario across multiple clusters simultaneously increases the risk of a widespread outage. Always start with a "canary" approach by selecting a single development cluster before expanding to a multi-cluster production-like environment.

- **Standardize Scenarios:** Use the same scenario version across all targets to ensure the results are comparable.
- **Verification:** Ensure that your recovery validation checks are defined globally, so you can clearly see when one cluster recovers slower than others, indicating a potential configuration issue in that specific region or environment.
- **Stop Conditions:** Configure strict stop conditions. If the experiment causes unintended degradation that exceeds your SLOs, ensure the "Stop on Failure" setting is enabled to rollback the scenario across all clusters immediately.

Navigating Chaos Studio and Resiliency Scores

Navigating Chaos Studio and Resiliency Scores

Chaos Studio is the central command center for orchestrating complex failure scenarios within Krkn. It moves beyond simple, single-pod disruption by providing a visual, node-based interface to design, execute, and analyze multi-stage resilience workflows.

Understanding Chaos Studio

Chaos Studio utilizes a graphical interface where every step of a failure injection is represented as a **node**.

- **Workflow Execution:** You can define workflows that execute scenarios in series (sequential) or parallel. This allows you to simulate cascading failures or simultaneous system stressors.
- **Graph Runs:** Every execution of a workflow is tracked as a "Graph Run." This provides a historical record where you can inspect the outcome, configuration, and logs for each individual node within the workflow independently.
- **Visual Validation:** By mapping nodes, you can visualize the dependency chain of your services and see how a disruption in one layer (e.g., node pressure) impacts higher-level application availability.

The Resiliency Score: Quantifying Stability

The Resiliency Score is a critical advancement over binary "pass/fail" chaos testing. It provides a numeric representation of how well your application maintained its Service Level Objectives (SLOs) while under duress.

How the Score Works

The Resiliency Score is not a static measurement; it is derived from live PromQL metrics defined by the user.

1. **Metric Definition:** You identify the key performance indicators (KPIs) that represent your system's health, such as API latency, HTTP 5xx error rates, or transaction throughput.
2. **PromQL Integration:** You define these metrics in a PromQL file, which is uploaded to Krkn via File Management.
3. **Real-time Evaluation:** During a Graph Run, Krkn executes these PromQL queries. The system compares the live metrics against the thresholds defined for your "steady state."
4. **Scoring:** The final score reflects the delta between your expected steady-state behavior and the behavior observed during the chaos injection.

Hands-on Activity: Configuring a Resiliency Score

In this activity, you will prepare a workflow to generate a quantitative resilience metric.

Prerequisites

- Access to the Krkn Operator dashboard.
- A pre-defined PromQL file containing your desired SLO metrics (e.g., `latency_threshold.yaml`).

Steps

1. **Access Chaos Studio:** Navigate to the **Chaos Studio** tab in your Krkn environment.
2. **Create/Edit Workflow:** Select your target workflow or create a new one by dragging scenario nodes onto the canvas.

3. **Enable Resiliency:** Within the workflow configuration panel, toggle the **Resiliency Score** option to "Enabled."
4. **Attach Metrics:**
 - Locate the **File Management** section in the UI.
 - Upload your PromQL configuration file if not already present.
 - In the workflow settings, mount the PromQL file to the execution environment.
5. **Execute:** Run the workflow. Once complete, navigate to the **Jobs** page.
6. **Analyze:** Click on your specific Graph Run. You will see the visual workflow alongside the calculated Resiliency Score. Compare the graph nodes against the spikes in your monitored metrics to identify which specific scenarios triggered the highest degradation in your score.

Best Practices for Interpretation

- **Define Baseline First:** Before relying on the Resiliency Score, ensure you have established a "steady state" baseline during normal operations. A score is only as accurate as the PromQL queries used to define your performance expectations.
- **Iterative Improvement:** Use low scores as a roadmap for architectural hardening. If a specific scenario causes a significant drop in the Resiliency Score, it indicates a lack of circuit breakers, inadequate replicas, or missing health check configurations in that service path.
- **Trend Analysis:** Do not look at scores in isolation. Compare scores across multiple Graph Runs to determine if recent code deployments have improved or regressed your platform's overall resilience.

Run and interpret one chaos experiment

Run and interpret one chaos experiment

Executing a chaos experiment involves transitioning from theoretical risk to actionable data by inducing controlled disruptions within a target cluster. The process focuses on validating system recovery and service reliability under stress.

- **Execution Setup:** Define the experiment parameters, select the target cluster, and utilize scenario registries to initiate the disruption.
- **Monitoring and Validation:** Continuously observe the system's steady state, focusing on key performance indicators like latency, throughput, and error rates.
- **Result Analysis:** Review experiment logs and status reports to determine if the system successfully recovered according to predefined criteria.
- **Impact Assessment:** Evaluate the effectiveness of configured alerts and assess how dependent components responded to the injected fault.
- **Blast Radius Management:** Ensure stop conditions are monitored during execution, with rollback plans ready to execute if the experiment exceeds defined safety thresholds.

Selecting target clusters and scenario registries

Selecting target clusters and scenario registries

In the Krkn ecosystem, the foundation of any chaos experiment lies in defining where the disruption will occur and which specific failure modes will be injected. Properly selecting target clusters and scenario registries ensures that your chaos engineering initiatives are both controlled and impactful.

Understanding Target Clusters

A "target cluster" is the Kubernetes environment where the Krkn chaos scenario will be executed. Krkn supports both single-cluster and multi-cluster targeting, allowing for localized testing or wide-scale stress testing of an entire fleet.

Key Considerations for Cluster Selection

- **Environment Parity:** Ensure the target cluster mirrors the configuration of your production environment to obtain meaningful data.
- **Blast Radius Management:** When selecting multiple clusters, remember that Krkn executes the scenario on all selected targets simultaneously. Verify that your system dependencies can withstand a broad-spectrum disruption.
- **Cluster Discovery:** Utilize integrations with Advanced Cluster Management (ACM) or Open Cluster Management (OCM) to dynamically discover and group clusters based on labels or health status.

Scenario Registries

Scenario registries act as the centralized repository for chaos scenarios. They contain the pre-defined logic (the "what" and "how") of the disruption, such as network latency, pod kills, or resource exhaustion.

- **Version Control:** Registries allow you to version your chaos experiments, ensuring consistency across different teams and environments.
- **Reusability:** By maintaining a standardized registry, you can ensure that the "Pod Disruption" scenario run in the development environment is identical to the one run in staging.
- **Abstraction:** Registries hide the underlying complexity of the Krkn disruption engine, allowing operators to select scenarios by name or capability rather than manual script configuration.

Hands-on Activity: Configuring Experiment Targets

This lab guides you through the process of selecting target clusters and choosing a scenario using the Krkn interface.

Prerequisites

- Access to the Krkn dashboard or CLI.
- Registered clusters accessible within the Krkn control plane.
- An active connection to the scenario registry.

Step 1: Selecting Target Clusters

1. Navigate to the **Run Scenario** wizard in your Krkn console.
2. In the **Target Selection** panel, view the list of discovered clusters.
3. Select the desired cluster(s) from the list.
 - **Tip:** Use the search or filter functionality if managing a large fleet of managed clusters.
4. Confirm the connection status for each selected cluster to ensure Krkn has the required permissions to execute pods within those namespaces.

Step 2: Selecting a Scenario from the Registry

1. Proceed to the **Scenario Selection** step.
2. Browse the available **Scenario Registries**. You will see categories like **network**, **pod-chaos**, or **node-disruption**.
3. Select a scenario (e.g., **pod-random-kill**).
4. Review the **Scenario Metadata** to understand:
 - **Description:** What the scenario specifically does.
 - **Parameters:** Editable fields such as **duration**, **percentage**, or **label_selector**.
5. Click **Next** to proceed to the configuration phase, where you will set the specific blast radius parameters for the selected target.

Troubleshooting

- **Cluster not appearing?** Verify that the Krkn operator has been correctly deployed and that the cluster is correctly registered with the ACM/OCM source.
- **Registry empty?** Check the configuration of your scenario registry URL. Ensure that your network allows egress traffic from the Krkn controller to the registry endpoint.

Configuring experiment parameters and attachments

Configuring experiment parameters and attachments

In Krkn, the precision of a chaos experiment relies entirely on how you define its parameters and manage its associated data. Proper configuration ensures that your disruption is targeted, measurable, and safe.

Understanding Parameter Categories

When configuring a scenario, parameters are organized into three distinct categories. You must address these to satisfy both the framework requirements and your specific testing goals:

- **Mandatory:** These are the "must-have" settings required for the scenario to function (e.g., target namespace, duration, or specific resource names). An experiment will not execute if these are missing.
- **Optional:** These provide fine-grained control over the behavior of the chaos. You use these to define label selectors, specific timing windows, or filters to narrow the scope of the disruption.
- **Global:** These are framework-level configurations. They govern how the experiment interacts with the broader ecosystem, such as integration settings for Prometheus, Elasticsearch, or Cerberus. You only modify these if you need to override the system defaults.

Managing File Attachments

Attachments in Krkn allow you to inject external logic or configuration into your experiments. Files are mounted at runtime and serve different purposes depending on where they are applied:

- **Run Scenarios:** Files are mounted as configuration files during the initial parameter setup to dictate the behavior of the scenario.
- **Chaos Studio (Nodes):** Files are mounted on individual nodes within a workflow. This allows you to apply unique configurations to specific steps in a multi-stage disruption.
- **Chaos Studio (Resiliency Score):** Specific files are mounted as **PromQL query files**. These define the mathematical logic used to calculate your Resiliency Score, allowing Krkn to evaluate system health before, during, and after the disruption.

Hands-on Lab: Configuring a Targeted Pod Disruption

In this lab, you will configure a Pod-kill experiment with specific parameters and a monitoring attachment.

Step 1: Scenario Parameterization

1. Navigate to the **Chaos Studio** interface or the scenario configuration YAML.
2. Select the **pod-chaos** scenario from the registry.
3. Set the **Mandatory** parameters:
 - **TARGET_NAMESPACE**: `production-app`
 - **DURATION**: `300s`
4. Define **Optional** parameters to limit the blast radius:
 - **LABEL_SELECTOR**: `app=frontend`
 - **POD_NAME_FILTER**: `frontend-*`

Step 2: Attaching the Resiliency Logic

To validate the experiment, you must attach the monitoring logic that defines your "Steady State." 1. Prepare a file named `frontend-availability.promql` containing your SLO query (e.g., `sum(rate(http_requests_total{status=~"2.."}[1m]))`). 2. In the **Resiliency Score** configuration section of your workflow node, upload or point to this `frontend-availability.promql` file. 3. This file will be mounted at execution time, allowing Krkn to compare real-time traffic against your baseline during the disruption.

Step 3: Preview and Execution

1. Navigate to the **Preview** tab in the configuration wizard.
2. Review the summary of your Mandatory, Optional, and Global parameters.
3. Ensure the attachment path for the PromQL file is correctly mapped.
4. Click **Execute** to trigger the experiment.



If you need to repeat a specific scenario with identical parameters at different stages of a workflow, use the **Clone** feature in Chaos Studio. Cloning duplicates the node configuration, saving time and ensuring consistency across complex multi-stage failure models.

Verification

Once the experiment completes, navigate to the **Logs and Status** report. Verify that: * The **TARGET_NAMESPACE** and **LABEL_SELECTOR** were respected. * The **Resiliency Score** calculation utilized your attached PromQL file. * No unexpected clusters or namespaces were impacted.

Reviewing configurations and executing experiments

Reviewing configurations and executing experiments

Before injecting disruptions into a production or staging environment, it is critical to perform a final validation of your experiment parameters. The Krkn Operator provides a structured preview mechanism to ensure that your "blast radius" is contained and your expected steady-state metrics

are clearly defined.

The importance of the pre-execution review

Executing a chaos experiment without a rigorous configuration review can lead to unintended consequences, such as exceeding the defined blast radius or triggering false-positive alerts. The review phase serves as the "stop condition" gatekeeper, allowing you to verify:

- **Target Accuracy:** Confirm the target cluster, namespace, and specific resource labels (e.g., `app=frontend`) match the intended scope.
- **Disruption Intensity:** Validate that parameters such as `duration`, `grace_period`, and `concurrency` align with your hypothesis.
- **Safety Mechanisms:** Ensure that your monitoring tools (Prometheus/Grafana) are configured to observe the target and that automated rollback or "stop" conditions are primed.

Steps to review and execute

When using the Krkn Operator interface, follow this systematic approach to transition from configuration to execution.

1. The Configuration Summary (Preview)

Before the `Run` command is finalized, the Krkn Operator generates a **Configuration Summary**. This interface aggregates all inputs provided in the scenario registry:

- **Scope Check:** Identify which clusters and nodes will be impacted.
- **Parameter Validation:** Review key-value pairs for the disruption (e.g., `pod_failure_percentage: 20`).
- **Dependency Check:** Verify that the required service account permissions and cluster discovery mappings are correctly resolved for the target cluster.

2. Executing the Experiment

Once the summary is validated, the execution process triggers the underlying Kubernetes Jobs.

- **Initialization:** Krkn Operator initializes the chaos engine pod within the target namespace.
- **Observability Sync:** The system begins streaming real-time status updates back to the Krkn dashboard.
- **Steady-State Monitoring:** The experiment begins by capturing the current state (Latency, Error Rate, Throughput) to establish a baseline before the disruption starts.

3. Monitoring Execution

During execution, utilize the Krkn terminal and logs interface: * **Real-time Logs:** Access the pod logs to verify that the scenario is interacting with the target resources as expected. * **Status Reports:** Track the progress percentage and current stage of the chaos workflow.



If at any point the system metrics deviate beyond your defined SLO thresholds,

trigger the manual abort mechanism via the Krkn dashboard to immediately revert the environment to a stable state.

Hands-on Activity: Executing a Pod-Disruption Experiment

In this activity, you will verify the configuration of a pod-disruption scenario and execute it against a test namespace.

Prerequisites: * Access to a Krkn-enabled cluster. * A target application deployed in a namespace (e.g., `demo-app`).

1. Step 1: Configuration Preview

- Navigate to the Krkn Operator **Run Scenarios** tab.
- Select the `pod-disruption` scenario.
- Configure the blast radius to target `namespace: demo-app`.
- Click **Preview** to load the summary modal.

2. Step 2: Validation

- Inspect the summary modal. Ensure the `target_label` matches your test application.
- Confirm that the `duration` is set to `5m` (5 minutes).

3. Step 3: Execution

- Click **Execute**.
- Navigate to the **Jobs** section to observe the pod spin-up.

4. Step 4: Observation

- Open your Prometheus dashboard or the Krkn **Resilience Score** view.
- Observe how the `error_rate` metrics shift as the pods are terminated and restarted by the Kubernetes controller.

5. Step 5: Post-Mortem

- Once the experiment completes, review the generated status report.
- Verify that the workload successfully recovered within the expected time frame.

Analyzing logs and status reports

Analyzing Logs and Status Reports

After executing a Krkn chaos experiment, the validation phase begins. Analyzing the results is critical to understanding how your system behaved during the disruption, whether your observability tools fired the correct alerts, and if the recovery process occurred within your defined Service Level Objectives (SLOs).

Understanding Krkn Execution Reports

When a Krkn experiment concludes, the Krkn Operator generates a comprehensive report that

summarizes the execution lifecycle. This report is your primary artifact for post-mortem analysis.

Component	Description
Status	Indicates the final outcome: Completed , Failed , or Aborted . A Failed status often points to a configuration error or a lost connection to the target cluster.
Duration	The total time elapsed during the disruption, including the stabilization period.
Log Aggregation	Provides access to the stdout of the scenario execution, detailing the specific disruption steps (e.g., specific pod kills or network latency injection).
Metrics Summary	Displays the captured state of your key indicators if integrated with your observability stack.

Interpreting Chaos Logs

Logs generated by Krkn provide granular visibility into the disruption sequence. When reviewing logs:

1. **Check Initialization:** Ensure the scenario correctly identified the target pods or nodes.
2. **Verify Disruption Timing:** Look for timestamps of when the chaos injection started and stopped. This is essential for correlating disruptions with application errors in your logs (e.g., ELK stack, Grafana Loki).
3. **Identify Failures during Injection:** If the scenario log shows **retries** or **timeouts** while attempting to impact a target, it may indicate that your blast radius is restricted by security policies or that the target resource was already unavailable.

Evaluating Status Reports and Observability

A chaos experiment is only as valuable as the data captured during its execution. To perform an effective analysis:

- **Correlate with Alerts:** Verify that your alerting system triggered for the correct services during the experiment. If you injected high latency but no latency-related alerts fired, your observability rules require tuning.
- **Compare Against Steady State:** Use the metrics gathered during the experiment to compare against the "steady state" defined before the test.
- **Validate Recovery:** Check if your system returned to normal operational parameters automatically without manual intervention.



If you have configured a **Resiliency Score** for your workflow, look at the scoring breakdown in the Jobs page. The score provides a summarized "health" trend based on the PromQL queries defined in your workflow, allowing you to quickly spot deviations from your performance thresholds.

Hands-on Activity: Analyzing Experiment Results

In this activity, you will review the output of a recently executed pod-disruption experiment.

1. **Locate the Job:** Navigate to the **Jobs** section in the Krkn dashboard.
2. **Select the Execution:** Click on the most recent completed experiment.
3. **Review Logs:**
 - Open the log view.
 - Search for the specific string identifying the affected pod (e.g., **Killing pod:** <pod-name>).
 - Verify the time taken for the **Ready** state of the pod to return to **True** post-disruption.
4. **Analyze Metrics:**
 - Open the **Metrics** tab associated with the job.
 - Compare the latency graph against the start/stop time found in your logs.
 - Determine if there was a spike that exceeded your pre-defined threshold.
5. **Documentation:** Record whether the system recovery was successful and if the alert duration matched the disruption duration.

If your analysis shows that recovery took longer than expected, document the bottleneck (e.g., slow container startup time or inefficient health checks) and propose a remediation plan for the next iteration.

Validating workload recovery and service availability

Validating Workload Recovery and Service Availability

After injecting a disruption using Krkn, the most critical phase of the chaos engineering lifecycle is evaluating how your system responds. Validation ensures that the "steady state" is restored and that your automated remediation strategies function as expected.

Understanding Recovery Validation

Validation is the process of confirming that your services return to their expected performance levels—Service Level Indicators (SLIs)—after a disruption. Relying on "self-healing" mechanisms without active verification is a common operational pitfall.

To validate recovery, you must monitor three distinct phases: 1. **Disruption Phase:** The period where the Krkn scenario (e.g., pod kill or network latency) is active. 2. **Impact Phase:** The duration where metrics deviate from the established baseline. 3. **Recovery Phase:** The duration from the cessation of the disruption until metrics return to the defined steady state.

Defining Success Criteria

Before executing a disruption, define what "recovery" looks like. Use your established metrics (latency, error rates, throughput) to create validation gates:

- **Self-Healing Verification:** Ensure that Machine Health Checks have successfully remediated unhealthy nodes or that the Kubernetes scheduler has successfully restarted pods.
- **Service Level Objective (SLO) Adherence:** Confirm that even during recovery, the impact on user-facing latency remained within the defined SLO buffers.
- **Data Integrity:** Verify that during the recovery phase, no data was corrupted or lost due to abrupt process termination.

Hands-on Lab: Validating Pod-Disruption Recovery

In this lab, you will use Krkn to disrupt a deployment and validate that the application recovers without human intervention.

Prerequisites

- A running Kubernetes cluster with Krkn Operator installed.
- A target deployment (e.g., `nginx-deployment`) with at least 3 replicas.
- Prometheus/Grafana configured to track the `kube_pod_status_ready` metric.

Step 1: Establish the Steady State

Execute a PromQL query to ensure the service is stable before starting.

```
sum(kube_pod_status_ready{namespace="my-app", condition="true"})
```

Ensure this returns a value of 3 before proceeding.

Step 2: Inject the Disruption

Create a Krkn scenario configuration to kill pods within the `my-app` namespace.

```
scenario:
  name: pod-disruption
  duration: 60s
  namespace: my-app
  instance_count: 2
```

Step 3: Observe Recovery Metrics

Once the disruption starts, observe the recovery in your monitoring dashboard. Look for the following events: 1. **Detection:** The `kube_pod_status_ready` count drops to 1. 2. **Scheduling:** The controller detects the missing pods and creates new ones. 3. **Recovery:** The count returns to 3.

Step 4: Validate and Analyze

Compare the time taken for recovery against your expectations. If recovery takes longer than anticipated, evaluate the following:

- **Readiness Probes:** Were the readiness probes too strict, delaying the "Ready" status of new pods?
- **Resource Constraints:** Did the cluster have enough capacity to schedule new pods immediately, or was it blocked by resource starvation?
- **Automated Remediation:** Did the Machine Health Checks trigger as expected to replace faulty nodes?

Troubleshooting Recovery Failures

If your service fails to return to the steady state:

- **Check Pending Pods:** If pods are stuck in `Pending`, examine the events: `kubectl describe pod <pod-name>`. This often indicates a lack of cluster resources.
- **Review CrashLoopBackOff:** If pods are failing to start, check the logs for dependency issues: `kubectl logs <pod-name> --previous`.
- **Inspect Topology:** If using multi-zone clusters, verify if the disruption caused a zone-wide failure that prevented recovery due to cross-zone latency or policy restrictions.

Verifying alerts and dependent component impact

Verifying alerts and dependent component impact

In Chaos Engineering, the ultimate measure of a system's resilience is not just whether the system survives a disruption, but whether the observability stack correctly captures the event. Verifying that alerts fire as expected and that dependent components behave predictably is critical to reducing Mean Time to Recovery (MTTR).

The Role of Alerts in Chaos Experiments

When you inject a failure using Krkn, your observability layer (typically Prometheus and Alertmanager) serves as the primary feedback loop. If a disruption occurs and no alert is triggered, you have discovered a gap in your monitoring—a "silent failure" that is often more dangerous than the disruption itself.

Key considerations for alert verification:

- * **Severity Alignment:** Ensure that the injected chaos triggers alerts at the expected severity (e.g., Warning vs. Critical).
- * **Correlation:** Verify that the alerts map directly to the service or node impacted by the Krkn scenario.
- * **Notification Fidelity:** Confirm that alerts reach the correct team channels (Slack, PagerDuty, etc.) with relevant SOP links or runbooks attached.

Analyzing Dependent Component Impact

Complex microservices environments are rarely isolated. A failure in a "dependency" (e.g., a database, an authentication service, or an API gateway) often triggers a cascading effect.

Cascading Failure Indicators

When monitoring dependent components, look for:

- * **Latency Spikes:** Upstream services waiting on a degraded downstream dependency.
- * **Error Rate Increases:** HTTP 5xx responses originating from services that cannot reach their dependencies.
- * **Resource Exhaustion:** If a dependency fails, look for connection pooling issues or thread pool saturation in the calling service.



To truly validate system robustness, **run multiple chaos tests simultaneously**. As noted in our best practices, combining disruptions (e.g., IO stress on a database while inducing network latency on the application) allows you to observe how dependent components react when their "steady state" is challenged by multifaceted failures.

Hands-on Lab: Verifying Alerting and Dependency Impact

In this lab, we will induce a controlled failure and verify that the monitoring stack correctly identifies the impact on dependent workloads.

Prerequisites

- A running Krkn Operator on your cluster.
- Prometheus and Alertmanager configured with active notification policies.
- A test application with clear Prometheus metrics and defined alerts.

Step 1: Baseline Verification

1. Access your Prometheus dashboard.
2. Ensure no firing alerts exist for the target namespace.
3. Note the current throughput and latency metrics for the dependent application.

Step 2: Inject a Focused Disruption

Execute a Krkn scenario designed to impact a specific dependency. For example, a pod-disruption scenario:

```
# chaos-scenario.yaml
scenario: pod-disruption
duration: 300s
target:
  label_selector: "app=backend-database"
blast_radius: 1
```

Execute via `oc` or `kubectl` by applying the job manifest or using the Krkn CLI.

Step 3: Verify Alerts

1. Immediately monitor your Alertmanager dashboard.
2. **Check:** Did the expected alert fire? If not, investigate the Prometheus rule definitions.

3. **Check:** Is the alert description actionable? Does it point to the correct affected component?

Step 4: Analyze Dependent Component Impact

Run a PromQL query to observe the impact on the upstream application while the database is being disrupted:

```
# Check error rates for the dependent service
rate(http_requests_total{status=~"5..", service="frontend-app"}[1m])
```

1. Observe if the **frontend-app** shows an increase in 5xx errors.
2. Confirm if the system auto-heals (recovery validation) once the Krkn job completes and the target pods are recreated.

Step 5: Post-Experiment Review

- **Document:** Record whether the alert fired within the expected timeframe.
- **Refine:** If the alert fired too late, adjust the **for** duration in your Prometheus alert rules.
- **Remediate:** If dependencies failed catastrophically rather than gracefully, implement circuit breakers or retries in the dependent application code.

Build workflows with Chaos Studio

Build workflows with Chaos Studio

Chaos Studio is a visual drag-and-drop editor designed to create complex, multi-stage resilience experiments. It allows you to model real-world infrastructure and service disruptions by defining sequences of events.

Key capabilities include:

- **Visual Workflow Design:** Construct graphs of chaos scenarios using a drag-and-drop interface.
- **Execution Control:** Configure scenarios to run in either serial (sequential) or parallel modes to simulate complex failure patterns.
- **Workflow Reusability:** Create, save, and clone workflow models to ensure consistent testing across different environments.
- **Resiliency Measurement:** Integrate workflows with the Resiliency Score to quantify how effectively your system withstands orchestrated disruptions.
- **Complex Modeling:** Map multifaceted failure modes into structured workflows to validate system recovery and SLO compliance.

Creating reusable chaos workflows

Creating reusable chaos workflows

Chaos Studio provides a visual, low-code environment to orchestrate complex failure patterns. Instead of executing isolated scenarios, you can design, save, and reuse **Chaos Workflows** to simulate real-world outages—such as a cascading failure caused by a network partition followed by a node drain.

Understanding Workflow Architecture

A chaos workflow is a directed graph where nodes represent individual chaos scenarios and edges define the execution flow. By leveraging **Chaos Studio**, you can move beyond simple disruption to modeling complex, multi-layered "failure days."

Key components of a workflow include:

- * **Nodes:** Represent a specific Krkn scenario (e.g., `pod-disruption`, `network-chaos`, `node-cpu-hog`).
- * **Edges:** Define the execution dependency, enabling sequential (chained) or parallel (simultaneous) operations.
- * **Configuration Files:** Reusable YAML definitions that store experiment parameters (duration, blast radius, target clusters).

Designing for Reusability

To ensure your workflows provide long-term value, follow these design principles:

- **Modularize Scenarios:** Break down complex incidents into discrete, reusable scenarios. For example, create a "Database Latency" scenario that can be plugged into various platform-level

workflows.

- **Abstract Configurations:** Use **File Management** to store parameter sets. By decoupling the scenario logic from the specific target infrastructure, you can reuse the same workflow template across development, staging, and production environments by simply swapping the configuration file.
- **Define Consistent Steady-State Metrics:** Ensure every workflow node includes a baseline health check. This ensures that when a workflow is reused, the Resilience Score remains consistent and comparable.

Practical Activity: Building a Multi-Stage Failure Workflow

In this lab, you will create a reusable workflow that simulates a two-stage failure: first a network latency event, followed by a controlled pod-disruption experiment.

Prerequisites

- Access to the Chaos Studio dashboard.
- A pre-configured scenario registry connected to your cluster.

Step 1: Initialize the Workflow Canvas

1. Navigate to **Chaos Studio** in your management console.
2. Click **Create New Workflow**.
3. Provide a name, such as `app-resilience-load-test`.

Step 2: Add Sequential Scenario Nodes

1. Drag the **Network Chaos** node onto the canvas.
2. Open the configuration panel for this node and select a pre-uploaded configuration file from the **File Management** library.
3. Drag a **Pod Disruption** node onto the canvas.
4. Connect the output of the **Network Chaos** node to the input of the **Pod Disruption** node. This establishes a **sequential execution** pattern.

Step 3: Configure Parallel Execution (Optional)

If you wish to test system recovery under simultaneous pressure: 1. Add two nodes: **Node Memory Stress** and **Node CPU Stress**. 2. Connect both nodes to the same preceding step. 3. Chaos Studio will execute these in parallel, allowing you to measure the cumulative impact on your application's throughput.

Step 4: Validate and Save

1. Click **Validate Workflow**. The engine will verify that all required parameters are defined in the associated configuration files.
2. Click **Save as Template**. This enables the workflow to appear in your "Reusable Templates" library for future use across different clusters.

Best Practices for Workflow Management

- **Versioning:** Always tag your workflow versions in your repository (e.g., `v1.0-network-failure-baseline`) to track how resilience improvements change the outcome over time.
- **Rollback Planning:** When building workflows, ensure the final node in a complex chain includes a "Health Recovery" check to verify the system has returned to its steady state after the chaos concludes.
- **Monitoring Integration:** Map your workflow stages to specific PromQL queries in your monitoring tool to visualize exactly when the **Resilience Score** drops during the workflow execution.

Configuring sequential and parallel scenario execution

Configuring Sequential and Parallel Scenario Execution

In complex distributed systems, failures rarely occur in isolation. To accurately simulate real-world outages—such as a cascading failure caused by a network partition followed by a service crash—Krk'n's Chaos Studio allows you to design sophisticated workflows. By chaining scenarios together, you can orchestrate complex failure patterns using sequential and parallel execution logic.

Understanding Workflow Execution Models

Chaos Studio workflows are built using a node-based architecture. Each node represents a specific disruption scenario. The way these nodes are connected defines the flow of the experiment:

- **Sequential Execution:** Nodes are linked in a linear path. A downstream node only initiates once the upstream node has successfully completed. This is ideal for testing cumulative system stress or multi-stage recovery processes.
- **Parallel Execution:** Multiple nodes are triggered simultaneously. This allows you to simulate "storm" conditions where multiple components fail at the exact same time, testing the system's ability to handle aggregate resource contention or simultaneous service unavailability.

Designing Workflow Logic

When building your workflow, consider the relationship between your disruptions:

1. **Sequential (Cascading):** Use this to test the "fail-over" capability. For example, trigger a `pod-kill` on a primary service, wait for the recovery, then trigger a `network-latency` event to see if the system recovers correctly under degraded networking.
2. **Parallel (Coincident):** Use this to test "blast radius" impact. For example, executing `memory-hog` on multiple nodes in different availability zones simultaneously to observe how the cluster scheduler (like Kubernetes) handles resource pressure across the platform.

Hands-on Lab: Creating a Multi-Stage Failure Workflow

In this lab, you will configure a workflow that tests system resilience against a simultaneous disruption followed by a secondary cascading failure.

Prerequisites

- Access to a Krkn-enabled cluster with the Chaos Studio operator installed.
- At least two scenario definitions registered in your registry.

Step 1: Initialize the Workflow

1. Navigate to the **Chaos Studio** dashboard in your Krkn UI.
2. Click **Create New Workflow**.
3. Provide a unique name and description (e.g., `resilience-stress-test-01`).

Step 2: Configure Parallel Disruption

1. Add two **Scenario Nodes** to the canvas.
2. Configure both nodes to run the `pod-disruption` scenario.
3. Drag the start-trigger point of both nodes from the **Start** node simultaneously. This ensures that the two pod disruptions begin at the exact same time.

Step 3: Configure Sequential Recovery Validation

1. Add a third node: **Health Check** or a secondary disruption like `network-partition`.
2. Connect the output of the two parallel `pod-disruption` nodes to the input of this third node.
3. **Logic Note:** By connecting them this way, the third node will wait for the parallel disruptions to conclude before initiating its own sequence.

Step 4: Validate and Execute

1. Click the **Validate Workflow** button to ensure no nodes are orphaned and configurations are valid.
2. Select your **Target Clusters** from the deployment dropdown.
3. Click **Run**.
4. Monitor the **Jobs** tab to observe the simultaneous start of the first two nodes, followed by the serial execution of the third.

Best Practices for Workflow Design

- **Define Clear Stop Conditions:** Always attach a "Stop Condition" to your workflow. If a parallel execution causes a breach of your SLOs (e.g., latency spikes above 500ms), the workflow should trigger an automatic rollback.
- **Keep it Modular:** Reuse workflow models. If you have a specific "Platform Stress" sequence, save it as a template to maintain consistency across different development and staging environments.

- **Analyze Dependencies:** Before creating parallel executions, verify that the services being targeted are not hard-dependent on each other in a way that would trigger a total cluster collapse, unless that is specifically the scope of your experiment.

Connecting and cloning workflow nodes

Connecting and cloning workflow nodes

In the Krkn Chaos Studio, complex failure scenarios are rarely the result of a single disruption. To model realistic platform-level failures—such as a cascading network partition followed by a pod-killer event—you must orchestrate multiple scenarios using the workflow canvas.

This section covers how to manipulate workflow nodes to build, expand, and structure your chaos experiments.

Visualizing Workflow Nodes

Within the Chaos Studio editor, every node functions as an individual Krkn scenario. A node is only considered valid for execution when all mandatory parameters, registry selections, and configuration files are properly defined.

- **Valid Nodes:** Displayed with green borders and green connection arrows.
- **Unconfigured Nodes:** Displayed with red borders and red connection arrows.



You cannot trigger a workflow execution if any node in the chain remains unconfigured. Always verify the state of your nodes before initiating an experiment.

Connecting Nodes for Execution Orchestration

The visual editor uses directional arrows to establish dependencies and dictate the order of operations. By connecting nodes, you define the "Chaos Pipeline."

Defining Execution Logic

- **Serial Execution:** Drag an arrow from the output port of one node to the input port of the next. Krkn will execute these scenarios sequentially; the second node will only begin after the first node has completed its disruption cycle.
- **Parallel Execution:** To trigger multiple failures simultaneously, branch your workflow by drawing arrows from a single source node to multiple downstream target nodes. This is highly effective for testing how a system handles "compounding" stress from different vectors at the same time.



When modeling complex disruptions, use parallel execution to simulate "noisy neighbor" scenarios or multi-vector attacks, ensuring your monitoring tools catch the aggregate impact on your SLOs.

Cloning Nodes for Efficiency

Manual configuration of recurring scenario parameters can be time-consuming, especially when testing the same failure mode with slight variations (or repeating the same event across different time intervals).

The **Clone** feature allows you to duplicate a fully configured node instantly.

- **Duplicate Configuration:** When you clone a node, all selected registries, scenario parameters, and mounted files are copied to the new instance.
- **Rapid Iteration:** Cloning is the preferred method for building "wave" disruptions. For example, if you need to run a **pod-disruption** scenario three times during a single workflow, configure it once, clone it twice, and chain them together sequentially.

Hands-on Activity: Modeling a Sequential Failure

In this activity, you will create a two-stage failure workflow using node cloning.

1. Create the Primary Node:

- Drag a new node onto the canvas.
- Select your registry and choose the **pod-disruption** scenario.
- Configure your mandatory parameters (e.g., **label_selector**, **namespace**).

2. Clone the Scenario:

- Right-click the configured node and select **Clone**.
- Place the second node to the right of the first.

3. Establish Dependency:

- Click the output port of the first node.
- Drag the arrow to the input port of the cloned node.

4. Validate:

- Observe the arrow color. If both nodes are green, the connection is valid.
- Click the **Execute** button to run the sequential disruption.



Always review the blast radius of cloned nodes. Even if parameters are identical, ensure that the target cluster settings for the cloned node match your intended scope to avoid accidental disruption outside of your test environment.

Validating and reusing workflow models

Validating and Reusing Workflow Models

In Chaos Engineering, the efficiency of your resilience program relies on the ability to define reliable, repeatable, and scalable experiments. Chaos Studio provides a visual environment not only for creating these experiments but also for ensuring they are production-ready through

rigorous validation and component reuse.

Validating Chaos Workflows

Before an experiment is executed, the Chaos Studio editor performs a real-time validation of your workflow graph. This step is critical to prevent mid-execution failures caused by misconfiguration.

The editor uses visual cues to communicate the readiness of your workflow:

- **Green Indicators:** When nodes are fully configured—meaning a registry is selected, the scenario is defined, and all mandatory parameters are populated—the connections (arrows) between nodes turn green. This indicates that the workflow is ready for execution.
- **Red Indicators:** If a node lacks mandatory parameters, is missing a file mount, or has an invalid connection, it will be highlighted with a red border and red connection arrows. A workflow cannot be triggered until all red indicators are resolved.



Only fully configured nodes can be connected to other nodes within a workflow. Always verify that your dependency chain is intact before attempting to execute a complex parallel or serial graph.

Reusing Workflow Components

To scale your testing strategy across different environments (e.g., development, staging, and production), you should adopt a "configure once, run anywhere" approach using the following mechanisms:

Node Cloning

Instead of manually recreating identical disruption patterns, use the **Clone** function within Chaos Studio. Cloning allows you to duplicate a fully configured node, including its parameters and file mounts. This is particularly useful for:

- * Repeating the same disruption scenario across different namespaces or deployments within a single workflow.
- * Standardizing the input parameters for identical scenarios used in different workflow branches.

File Management

Chaos Studio integrates a centralized **File Management** system to handle external dependencies. You can upload and manage text-based configuration files (such as custom `kubeconfig` files, specialized scenario scripts, or disruption parameter overrides) within the platform.

- * **Decoupling:** By referencing these files rather than hardcoding values, you can swap out configurations (e.g., changing a target cluster's scale parameters) without modifying the workflow graph itself.
- * **Reusability:** Once a file is uploaded, it becomes available to any scenario or workflow node, ensuring consistency across your entire resilience suite.

Hands-on Lab: Constructing and Reusing a Workflow

In this lab, you will build a repeatable workflow and prepare it for reuse across clusters.

Step 1: Configure and Validate a Node

1. Open **Chaos Studio** and create a new workflow.
2. Drag a "Pod Disruption" scenario onto the canvas.
3. Click the node to open the configuration panel. Select your **Scenario Registry** and choose a **pod-delete** scenario.
4. Input the mandatory parameters (e.g., **namespace** and **label_selector**).
5. Observe the node status. If the arrows turn **green**, proceed. If **red**, check for missing mandatory fields.

Step 2: Implement Workflow Reuse

1. Once the first node is validated, select it and click **Clone**.
2. Place the cloned node in parallel to the first.
3. Upload a shared configuration file via the **File Management** tab containing specific resource thresholds.
4. Attach this file to both nodes.

Step 3: Final Validation

1. Connect the nodes into a parallel execution pattern.
2. Verify that all connections are green.
3. Save the workflow model. You can now import this model into other environments by simply updating the target registry references, ensuring your resilience testing methodology remains standardized across your organization.

Modeling complex platform and service disruptions

Modeling complex platform and service disruptions

In distributed systems, failures are rarely isolated events. A network partition may lead to a cascading failure of microservices, which in turn triggers a resource exhaustion scenario on your nodes. Modeling these **complex, multi-stage disruptions** is essential for validating the true resilience of your platform.

By moving beyond simple "kill-a-pod" scenarios, you can simulate realistic "worst-case" environments to observe how your system handles compound stress.

Understanding Multi-Stage Disruptions

A complex disruption model simulates a chain reaction. Instead of testing for a single point of failure, you test for:

- **Cascading failures:** How service A recovers when service B is latency-impacted and service C is unavailable.
- **Resource starvation loops:** Monitoring how node recovery (e.g., node reboot) interacts with application re-scheduling and pod readiness probes.
- **Platform-Application interaction:** Observing how cloud infrastructure instability (node termination) affects the Kubernetes control plane and subsequent application health.

Strategy: The "Blast Radius" Chain

To model these effectively in Krkn, we use the following methodology:

1. **Define Dependencies:** Map out which services rely on others.
2. **Sequence the Chaos:** Use workflow orchestration to execute disruptions in a specific order (e.g., first introduce network latency, then terminate a node).
3. **Validate Steady State Transitions:** Each stage of the disruption must have a defined expectation for metrics (latency/throughput) before moving to the next stage.

Hands-on Activity: Modeling a Compound Failure

This activity demonstrates how to chain a **pod-disruption** with a **node-disruption** using Krkn's workflow capabilities.

Prerequisites

- Krkn Operator installed and configured in your target cluster.
- Access to the Chaos Studio workflow editor.

Step 1: Define the Workflow Model

Create a new workflow configuration file (**complex-failure-workflow.yaml**). You will define two sequential nodes.

```
name: "node-and-pod-stress-test"
nodes:
  - name: "stage-1-pod-kill"
    scenario: "pod-scenarios"
    parameters:
      label_selector: "app=critical-service"
      count: 2
  - name: "stage-2-node-termination"
    scenario: "node-scenarios"
    depends_on: "stage-1-pod-kill" # Wait for pods to restart before hitting the node
    parameters:
      action: "terminate"
```

```
provider: "aws"
```

Step 2: Configure Stop Conditions

Crucially, add "circuit breakers" to your model. If your service availability drops below a defined threshold, the workflow must halt to prevent unintended outages.

- **PromQL Check:** Set a monitor for 5xx errors.
- **Stop Condition:** If `rate(http_requests_total{status="5xx"}[1m]) > 5%`, trigger an immediate rollback.

Step 3: Execute and Observe

1. Navigate to **Chaos Studio** in your Krkn dashboard.
2. Import your `complex-failure-workflow.yaml`.
3. Execute the workflow and monitor the **Resilience Score** dashboard.
4. Observe the transition:
 - **Phase 1:** Are the application replicas surviving the initial pod kills?
 - **Phase 2:** Once the node is terminated, does the cluster scheduler trigger `MachineHealthChecks` or pod re-scheduling effectively?

Best Practices for Complex Modeling

- **Start with "Steady State":** Never run complex scenarios without first establishing a baseline for your KPIs (latency, error rates) under normal load.
- **Validate Recovery:** Ensure your `PodDisruptionBudgets` (PDBs) are strictly defined. For example, if you are testing node failure, ensure your PDBs prevent the cluster from evicting all replicas of an `etcd` or `zookeeper` cluster simultaneously.
- **Analyze the "Tail":** Focus on the p99 latency during the transition phase. This is often where cascading failures manifest before a total outage occurs.



When modeling platform disruptions, always include a "wait" period between scenario stages. This allows the system's automated recovery processes (like Kubelet restarts or Load Balancer health checks) to attempt remediation before you apply the next layer of chaos.

Measure resilience and establish a baseline

Measure resilience and establish a baseline

Measuring resilience shifts testing from binary pass/fail outcomes to data-driven insights. By establishing a baseline of normal operation, you can quantify how effectively a system maintains performance under stress.

- **PromQL Integration:** Utilize PromQL files to define and monitor critical metrics such as latency, error rates, and throughput against predefined thresholds.
- **Resiliency Score Calculation:** Generate a numeric Resilience Score by evaluating application metrics throughout a chaos workflow, providing a quantifiable measure of how well the system withstood disruption.
- **Performance Benchmarking:** Compare scores across different environments and time periods to track resilience trends and identify performance degradation relative to Service Level Objectives (SLOs).
- **Evidence-based Remediation:** Use objective score analysis to pinpoint architectural weaknesses and provide actionable recommendations for system hardening.

Using PromQL for SLO definition

Using PromQL for SLO definition

In the context of Chaos Engineering with Krkn, defining a Service Level Objective (SLO) is the transition from "chaos for the sake of chaos" to "data-driven resilience validation." By leveraging PromQL (Prometheus Query Language), you transform abstract system health into quantitative benchmarks that determine whether your application successfully withstood a disruption.

The Role of PromQL in Resiliency

Krkn utilizes PromQL to evaluate the "Steady State" of your services. Rather than relying on simple binary checks (e.g., "Is the pod running?"), PromQL allows you to measure the actual user experience and system performance during a chaos experiment.

When defining an SLO for a Krkn experiment, you focus on three primary dimensions: * **Availability:** The percentage of successful requests versus total requests. * **Latency:** The duration taken to process requests (e.g., P99 latency). * **Throughput:** The volume of requests processed per unit of time.

Defining SLOs with PromQL

To integrate SLOs into your chaos workflows, you must define PromQL expressions that represent your "normal" operational boundaries.

Example: Latency SLO

A typical latency SLO might define that 99% of requests must be served in under 200ms. In PromQL, this would look like:

```
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) < 0.2
```

If this value spikes above **0.2** during a Krkn scenario (such as a pod-kill or network latency injection), the Resilience Score calculation will penalize the result, indicating a breach of the SLO.

Example: Error Rate SLO

To ensure the application remains functional, you might define an error rate limit:

```
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100 < 1
```

This query triggers a failure in your resiliency assessment if the error rate exceeds 1% over a 5-minute window.

Implementing PromQL in Krkn Workflows

To evaluate these metrics during a Krkn execution, follow this integration pattern:

1. **Create a PromQL File:** Store your queries in a structured format (usually a YAML file containing key-value pairs of query names and expressions).
2. **File Management:** Upload this configuration file to the Krkn environment via the File Management interface.
3. **Workflow Configuration:** Enable the **Resiliency Score** option on your chaos workflow.
4. **Mounting:** Point the workflow to your uploaded PromQL file. During the experiment, Krkn will poll the in-cluster Prometheus instance using these queries.



Krkn uses the results of these queries to calculate the **Resiliency Score**. If a query result deviates from the threshold defined in your PromQL logic, Krkn marks the exit status accordingly. You can set severity levels for each query to distinguish between minor performance degradation and critical service failure.

Hands-on Lab: Defining and Testing an SLO

In this activity, you will define a basic SLO for a sample service and link it to a Krkn workflow.

- **Step 1:** Identify a key metric for your service (e.g., `container_cpu_usage_seconds_total`).
- **Step 2:** Write a PromQL expression that defines the "healthy" limit for this metric.
- **Step 3:** Save the expression into a `slos.yaml` file.
- **Example structure:** `[source,yaml] --- queries:`

- name: "High CPU Check" query: "sum(container_cpu_usage_seconds_total) < 0.5" severity: "critical" ----

- **Step 4:** Upload `slos.yaml` to the Krkn File Manager.
- **Step 5:** In your Chaos Workflow configuration, enable the **Resiliency Score** checkbox and select `slos.yaml` as your metric source.
- **Step 6:** Execute a disruption scenario (e.g., `pod-disruption`) and observe the **Jobs** dashboard to view the impact on your newly defined SLO.

By mapping your risks to these specific PromQL metrics, you move from manual observation to automated, objective-based resilience verification.

Monitoring availability, error rates, and throughput

Monitoring Availability, Error Rates, and Throughput

To effectively evaluate system resilience during Krkn chaos experiments, you must move beyond simple "up/down" checks. You must establish a baseline for your steady state by monitoring three fundamental pillars of service health: **Availability**, **Error Rates**, and **Throughput**.

Defining the Metrics

When injecting chaos, these metrics serve as your primary indicators for whether the system is successfully absorbing the disruption or if the user experience is suffering.

- **Availability:** The percentage of time a system is functional and reachable. In a Kubernetes environment, this is often measured by the success rate of HTTP requests or the ratio of healthy pods to total pods.
- **Error Rates:** The frequency of failed requests or internal server errors (e.g., HTTP 5xx codes). Spikes in error rates during a Krkn scenario usually indicate that the service or its dependencies are failing to handle the injected disruption.
- **Throughput:** The amount of work the system performs over a specific time, such as requests per second (RPS) or transactions per second (TPS). A drop in throughput often reveals bottlenecks in resource handling or queuing mechanisms.

Establishing a Baseline using PromQL

Before executing a Krkn scenario, use Prometheus Query Language (PromQL) to define your "normal" behavior. These queries will be used to generate alerts or stop conditions if the system deviates from your established baseline.

Availability Query

To monitor the availability of an HTTP service based on response codes:

```
(sum(rate(http_requests_total{status!~"5.."}[5m])) /  
sum(rate(http_requests_total[5m]))) * 100
```

Error Rate Query

To monitor the rate of 5xx errors:

```
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

Throughput Query

To monitor the requests per second hitting your application:

```
sum(rate(http_requests_total[5m]))
```

Hands-on Lab: Observing Metrics during Disruption

In this activity, you will monitor these metrics while Krkn executes a pod-disruption scenario.

Prerequisites

- A target cluster with **Krkn** installed.
- A sample application (e.g., **http-echo**) deployed with monitoring enabled.
- Access to a Grafana dashboard or Prometheus instance configured for the target cluster.

Step-by-Step Execution

1. **Establish Steady State:** Access your monitoring dashboard and observe the metrics for 5 minutes. Document the baseline throughput and verify that error rates are near zero.
2. **Initiate Krkn Scenario:** Execute a pod-disruption experiment using the Krkn scenario registry to terminate a percentage of your application pods.
 - **Example Command:** `krkn run pod-disruption --scenario-config=config.yaml`
3. **Real-Time Monitoring:** Observe the dashboard as the chaos is injected:
 - **Availability:** Does it dip? If so, by how much, and for how long?
 - **Error Rates:** Observe if the 5xx rate increases immediately upon pod termination.
 - **Throughput:** Look for a correlation between the pod outage and the drop in throughput.
4. **Analyze Recovery:** Once the Krkn scenario concludes and the pods are recreated, watch the metrics return to their pre-experiment baseline. This "recovery curve" is vital for calculating your system's resilience.

Troubleshooting Tips

- **False Positives:** Ensure your Prometheus scrape intervals are frequent enough (e.g., 15s) to capture the transient spikes during the chaos injection.
- **Metric Granularity:** If throughput is monitored at too high a level (e.g., cluster-wide), you may miss micro-outages at the individual service level. Ensure you apply labels (e.g., `service="my-app"`) to your PromQL queries.

- **Dependency Impact:** If error rates spike in a service that is **not** the target of the Krkn scenario, you have successfully identified a critical downstream dependency. Use this to inform your recovery validation planning.

Calculating and interpreting the Resilience Score

Calculating and interpreting the Resilience Score

The Resiliency Score is a quantitative metric provided by the Krkn Operator that evaluates how well your application maintains its service levels during a chaos experiment. Unlike binary pass/fail tests, the Resiliency Score provides a nuanced view of system stability by correlating chaos events with real-time application telemetry.

Understanding the Resiliency Score Mechanism

The score is derived from your application's Service Level Indicators (SLIs). By analyzing these metrics via PromQL during the chaos workflow, Krkn translates technical performance data into a normalized score.

Core Components

- **PromQL Definition:** A structured file containing queries for latency, error rates, or throughput that defines your "steady state."
- **Thresholds:** Expected performance ranges defined within your queries.
- **Workflow Integration:** The score is calculated by evaluating these queries across the duration of the Graph Run in Chaos Studio.

Configuring Resiliency Score Calculation

To calculate a Resiliency Score, you must define the metrics that matter most to your application's reliability.

1. **Define PromQL Queries:** Create a file that captures your key performance metrics.

```
# Example structure for a resilience_metrics.promql file
# Query for error rate threshold
sum(rate(http_requests_total{status=~"5.."}[1m])) /
sum(rate(http_requests_total[1m])) < 0.05
```

2. **Enable Resiliency Score in Chaos Studio:**

- Navigate to your workflow in **Chaos Studio**.
- Select the **Resiliency Score** option within the workflow configuration settings.
- Mount your **resilience_metrics.promql** file via the File Management interface.

3. **Execute and Monitor:**

- Run the workflow. The Krkn Operator will execute the mounted PromQL queries in parallel

with the chaos scenarios.

- Results are aggregated and processed to generate the final score, which appears in the **Jobs** page.

Interpreting the Results

Once the execution completes, the Jobs page displays the Resiliency Score alongside the Graph Run visualization.

- **Quantitative Insights:** A high score indicates that your application successfully maintained its expected performance (SLOs) despite the injected failures. A low score highlights specific areas where the system failed to recover or degraded beyond acceptable limits.
- **Granular Analysis:** Since the score is tied to the Graph Run, you can inspect individual nodes (scenarios) to see exactly which disruption caused a drop in the resilience score.
- **Trend Analysis:** By running identical workflows over time—especially after code changes or infrastructure upgrades—you can compare scores to determine if your application's resilience is improving or regressing.

Best Practices for Resilience Measurement

- **Map to SLOs:** Ensure that your PromQL queries reflect your business-critical Service Level Objectives. If latency spikes, the score should reflect the degradation of user experience.
- **Standardize Metrics:** Use the same PromQL file across different environments (e.g., staging vs. production) to establish a consistent baseline for resilience.
- **Iterative Remediation:** Do not treat the score as just a metric. If a chaos scenario causes a significant drop in the Resiliency Score, investigate the specific node logs, remediate the underlying recovery issue, and re-run the test to validate the fix.



The Resiliency Score is most effective when evaluated against a "Steady State." Ensure your baseline metrics are captured during normal operations before running your first high-impact chaos experiment.

Comparing scores across environments and time

Comparing scores across environments and time

To effectively build a culture of resiliency, a single Resiliency Score is merely a snapshot. The true value of Chaos Engineering with Krkn lies in longitudinal analysis—comparing your system's performance across different environments (e.g., Staging vs. Production) and tracking how your resiliency evolves over time as your architecture matures.

The Importance of Comparative Analysis

Comparing Resiliency Scores serves two primary purposes:

- **Environment Parity Validation:** By running identical chaos scenarios in Staging and Production, you can verify if your configuration, auto-scaling policies, and recovery

mechanisms perform consistently. Discrepancies often highlight "hidden" production configurations that are missing in lower environments.

- **Trend Analysis:** Tracking scores over time helps you quantify the impact of architectural changes. If a score trends downward following a cluster upgrade or a new service deployment, you have immediate empirical evidence of regression in system resilience.

Strategies for Comparative Benchmarking

To ensure your comparisons are scientifically valid, adhere to these practices:

1. Standardize the PromQL Baseline

The Resiliency Score is only as accurate as the PromQL queries defined for your steady state. When comparing environments, ensure you are using the **same PromQL file** and the **same expectation thresholds**. If the thresholds differ, the "score" will reflect the variance in expectation rather than the variance in system stability.

2. Control for Load Variance

Resiliency scores can be highly dependent on the traffic volume at the time of the experiment. * **Observation:** A system might appear resilient during low-traffic periods but fail under peak load. * **Action:** When comparing scores across time, correlate them with traffic intensity metadata. Krkn allows you to capture throughput metrics; use these as context when evaluating if a score improvement is due to better code or simply lower traffic.

3. Mapping Scores to Version Metadata

Always tag your experiment runs with version identifiers (e.g., Git commit hash, application version, or infrastructure release version). This allows you to create a timeline of resilience that maps directly to your deployment pipeline.

Evaluating Trends in Krkn

When analyzing your history in the Krkn dashboard, look for the following patterns:

Pattern	Interpretation	Recommended Action	:--	:--	:--	Consistent High Score
Mature system with robust self-healing.	Maintain existing tests; focus on testing "unknown unknowns."		Downward Trend	Regression introduced in recent deployments.	Audit recent changes; increase frequency of automated chaos tests.	High Variance
Unstable recovery or intermittent dependencies.	Investigate "flaky" components; check for resource contention.		Score Plateau	System has reached its current resiliency limit.	Review architecture; consider adding new redundancy or failover patterns.	

Operationalizing Historical Review

To make these comparisons a standard part of your workflow:

1. **Centralize Reporting:** Use the Krkn reporting interface to export results into a time-series database. This allows for long-term visualization beyond the default retention period of the Krkn operator.

2. **Conduct Quarterly Resiliency Reviews:** Periodically aggregate scores from your staging and production environments to identify systemic areas of weakness that were not captured by individual experiment alerts.
3. **Correlation with SLOs:** If your Resiliency Score is consistently high but your real-world service SLOs are dropping, re-evaluate your PromQL metrics. It indicates that your chaos experiment is not correctly modeling the "blast radius" or the specific failure modes affecting your users.



When you notice a significant shift in scores across time, use the **Resiliency Score history** in the Krkn UI to isolate the specific "Jobs" that contributed to the score drop. Check if the underlying PromQL metric threshold was breached (e.g., latency spikes exceeding the 500ms limit during a node-kill experiment).

Identifying SLO degradation and performance trends

Identifying SLO degradation and performance trends

In the context of Chaos Engineering with Krkn, identifying Service Level Objective (SLO) degradation is the primary indicator of your system's resilience. While chaos experiments intentionally disrupt components, the goal is to determine if your architecture and self-healing mechanisms maintain performance within predefined thresholds.

Understanding SLO Degradation

SLO degradation occurs when the metrics tracking your service's health drift outside of agreed-upon boundaries during or after a disruption.

To effectively identify these trends, you must distinguish between: * **Expected Transient Impact:** Short-lived latency spikes caused by node restarts or pod evictions where the system self-heals within defined recovery time objectives (RTO). * **Systemic SLO Violation:** Sustained degradation where throughput drops or error rates climb, signaling that the system's redundancy or load balancing is insufficient to handle the injected failure.

Monitoring Performance Trends

To spot degradation, you must baseline your system's "Steady State" using PromQL queries. When running Krkn experiments, compare these metrics across three distinct phases:

1. **Pre-Disruption (Baseline):** Your "normal" operating window.
2. **Disruption (Turbulence):** The period during which Krkn is executing its scenario (e.g., pod kills, network latency injection).
3. **Post-Disruption (Recovery):** The period following the removal of the disruption to see if metrics return to the baseline or remain degraded.

Monitoring Metrics with PromQL

Use the following patterns in your observability platform (e.g., Grafana/Thanos) to visualize degradation:

- **Availability (Success Rate):** `sum(rate(http_requests_total{status=~"2.."}[5m])) / sum(rate(http_requests_total[5m]))`
- **Latency (p99):** `histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))`
- **Throughput:** `sum(rate(http_requests_total[5m]))`

Analyzing Performance Trends

When reviewing the results of a Krkn experiment, look for the following patterns to identify performance degradation:

- **The "Limping" Pattern:** Metrics do not return to the pre-disruption baseline, indicating that the system's automated recovery failed to restore the original state (e.g., a node failed to rejoin the cluster, or a process is stuck in a crash loop).
- **Cascading Failure:** An SLO violation in one service triggers a chain reaction in dependent components, leading to a wider outage than the initial blast radius intended.
- **Slow Recovery:** The system eventually recovers, but the time taken exceeds your Recovery Time Objective (RTO).

Hands-on Lab: Detecting SLO Degradation

In this exercise, we will observe how an SLO threshold reacts to a pod-disruption experiment.

1. **Step 1: Define the Threshold** Identify a critical latency metric. For example, if your p99 latency baseline is `200ms`, set your observation threshold to `300ms` as a trigger for "Degraded."
2. **Step 2: Execute Krkn Experiment** Run a `pod-disruption` scenario using the Krkn Operator:


```
[source,yaml] --- apiVersion: chaos.krkn.cloud/v1beta1 kind: ChaosScenario metadata: name: pod-disruption-test spec: scenario: pod-disruption parameters:
```

 - `name: EXPECTED_RECOVERY_TIME value: "60" ----`
3. **Step 3: Correlate and Visualize** Observe your dashboard during the experiment.
 - If latency exceeds `300ms` for longer than the `EXPECTED_RECOVERY_TIME`, log the event as a **Degraded State**.
 - Compare this timestamp against the Krkn execution logs to confirm if the degradation was a direct result of the chaos injection or an underlying platform instability.
4. **Step 4: Remediation Recommendation** If the trend shows consistent SLO degradation during specific failure modes:
 - **Review Pod Disruption Budgets (PDBs):** Are there enough replicas to satisfy the disruption?
 - **Optimize Readiness Probes:** Is the application taking too long to become "Ready" after a restart?
 - **Scaling:** Does the cluster need horizontal autoscaling to handle the diverted traffic during the disruption?

Recommending remediation based on score analysis

Recommending Remediation Based on Score Analysis

After executing a chaos experiment using Krkn, the resulting **Resiliency Score** acts as a diagnostic tool rather than just a final grade. A low score or a degradation in service metrics during a disruption identifies specific architectural weaknesses.

This section guides you through interpreting these scores to develop actionable remediation strategies.

Analyzing Score Degradation

The Resiliency Score is derived from the PromQL queries defined in your experiment configuration. When the score drops, it indicates that your application's Service Level Indicators (SLIs)—such as latency, error rates, or throughput—deviated from the expected steady state during the chaos injection.

To effectively recommend remediation, follow this investigative process:

1. **Correlate Timeframes:** Use the "Jobs" page in Chaos Studio to overlay your disruption timeline with the metrics captured in your PromQL file.
2. **Identify the "Breaking Point":** Determine if the degradation began immediately upon fault injection or if it occurred during the recovery phase.
3. **Evaluate Dependency Impact:** If the cluster health signal (e.g., Cerberus) dropped, the issue likely resides in infrastructure dependencies or service mesh configurations rather than the application code itself.

Categorizing Remediation Strategies

Based on your score analysis, remediation typically falls into one of three categories:

Category	Remediation Approach
Resource Tuning	If metrics show resource exhaustion (CPU/Memory throttling), adjust Kubernetes HPA/VPA settings or increase requested/limit resources.
Architectural Resilience	If the score indicates service unavailability due to pod disruption, implement Pod Disruption Budgets (PDBs), anti-affinity rules, or increase replica counts.
Observability/Alerting	If the system recovered but the score remains low due to hidden errors, update your alerts or probes to detect failure modes faster.

Practical Step: Moving from Score to Action

Follow these steps to generate a remediation plan based on your experiment results:

1. **Extract Metric Anomalies:** Open the Prometheus UI or your dashboard and query the metrics used in your PromQL file for the specific duration of the chaos experiment. **Example Query:**

[source,promql] ---- sum(rate(http_requests_total{status=~"5.."}[1m])) /
sum(rate(http_requests_total[1m])) ---- If this error rate exceeds your threshold during the
experiment, your remediation target is service availability.

2. **Implement Circuit Breakers or Timeouts:** If the score analysis shows that a single failing dependency caused a "cascading failure" (where the application waits indefinitely for a non-responsive service), recommend implementing circuit breakers in your service mesh or application logic.
3. **Refine Recovery Validation:** If the application eventually recovered but took too long, the remediation may not be the application itself, but the **readiness probes**. Update probes to be more aggressive to ensure that traffic is routed away from unstable pods faster.
4. **Iterate and Re-test:** After applying the recommended changes, execute the same chaos scenario again. A successful remediation should result in a higher Resiliency Score, demonstrating that the system's "Steady State" was maintained despite the disruption.

Troubleshooting Guidance

If you find that the score does not improve after remediation, consider the following: * **Scope Creep:** Did the blast radius expand during the fix? Verify that your experiment is still targeting the exact same components. * **Metric Sensitivity:** Is your PromQL threshold too strict? If your system is technically resilient but the score is low, review the baseline expectations defined in your PromQL file to ensure they reflect realistic operational limits. * **External Noise:** Ensure no other deployments or performance tests were running simultaneously that could have skewed the baseline metrics used for the calculation.

Operationalize resilience validation

Operationalize resilience validation

Operationalizing resilience validation involves integrating chaos engineering into the software development lifecycle to ensure continuous system health and reliability. This practice moves beyond one-off tests by standardizing how resilience is measured and maintained across environments.

Key aspects include:

- **CI/CD Integration:** Embedding chaos experiments directly into delivery pipelines to validate infrastructure and application upgrades automatically.
- **Standardization:** Implementing consistent execution patterns and recovery checks across all managed clusters.
- **Performance Context:** Running chaos tests during scale and performance testing to observe behavior under high load.
- **Long-term Analysis:** Storing, comparing, and trending results over time to identify degradation patterns and ensure SLO compliance.
- **Holistic Monitoring:** Combining automated recovery validation with system health metrics to provide a comprehensive view of service resilience.

Validating infrastructure and application upgrades

Validating Infrastructure and Application Upgrades with Chaos Engineering

In a cloud-native ecosystem, upgrades—whether at the infrastructure level (e.g., Kubernetes version upgrades, node OS patching) or the application level (e.g., deployment rollouts)—are frequent sources of instability. Traditional testing often focuses on functional success, but fails to account for the "turbulent" conditions that occur during transition states.

By integrating Krkn into your upgrade lifecycle, you can move beyond simple health checks to **Resilience Validation**, ensuring that your systems not only survive the update process but maintain service level indicators (SLIs) throughout.

The Role of Chaos in Upgrades

Upgrades often involve shifting traffic, draining nodes, or restarting pods. If these processes encounter unforeseen issues, the result is often cascading failure. Chaos engineering allows you to simulate "worst-case" scenarios during these transitions:

- **Infrastructure Upgrades:** Inject network latency or pod disruptions during a node drain to ensure Pod Disruption Budgets (PDBs) are configured correctly and workloads rebalance without downtime.
- **Application Upgrades:** Trigger service failures while a rolling update is in progress to verify

that the system can handle concurrent versioning and potential compatibility gaps.

Strategy for Validation

To validate an upgrade, follow this structured approach:

1. **Define Steady State:** Establish baseline metrics for latency, error rates, and throughput before initiating the upgrade.
2. **Define Stop Conditions:** Configure automated triggers (e.g., Prometheus alerts) that halt the upgrade if latency exceeds a defined threshold during a chaos experiment.
3. **Run Concurrent Chaos:** Execute Krkn scenarios (such as `pod-scenarios`) simultaneously with your deployment or upgrade pipeline to test system adaptability.
4. **Observe Recovery:** Measure the Mean Time to Recovery (MTTR) as the system settles back into the new steady state.

Hands-on Lab: Validating a Rolling Update

In this lab, you will use Krkn to inject pod disruptions during a deployment rollout to ensure that your application remains available throughout the process.

Prerequisites

- A running Kubernetes cluster.
- Krkn Operator installed and configured.
- An application with a defined `Deployment` and `PodDisruptionBudget`.

Step 1: Establish the Baseline

Monitor your application's request latency using your monitoring stack (e.g., Grafana/Prometheus). Ensure the current version is stable.

Step 2: Configure the Chaos Experiment

Create a Krkn scenario file (`upgrade-chaos.yaml`) to target the application pods during the upgrade:

```
scenario: pod-scenarios
duration: 300
namespace: my-app-namespace
label_selector: "app=my-web-service"
actions:
  - action: kill_pods
    count: 1
    probability: 0.5
```

Step 3: Execute and Validate

1. **Initiate the Upgrade:** Start your application rollout (e.g., `kubectl set image deployment/my-app-web web=new-version`).

2. **Run the Experiment:** Execute the Krkn job simultaneously: `oc apply -f upgrade-chaos.yaml`

3. **Monitor Resilience:**

- Observe the `Replicas` and `AvailableReplicas` in your deployment.
- Check for spikes in 5xx error codes.
- Verify that your alerting system fires appropriately if thresholds are breached, but the upgrade completes successfully.

Step 4: Analyze and Remediate

If the service availability drops below your SLO during the combined upgrade and chaos run: *

Audit PDBs: Ensure `minAvailable` or `maxUnavailable` is set to allow for the concurrent upgrade and chaos-induced pod loss. * **Refine Readiness Probes:** Ensure readiness probes are accurate so the load balancer does not route traffic to pods that are in an unstable transition state.

Best Practices for Operationalizing

- **Automate in CI/CD:** Incorporate Krkn execution into your pipeline (e.g., Jenkins or Tekton) to trigger chaos tests automatically following the completion of an infrastructure update.
- **Standardize Metrics:** Use a consistent `Resilience Score` across different environment upgrades to track if your system becomes more or less resilient after specific patches.
- **Alert Consistency:** Ensure that chaos-induced events do not "alert fatigue" your SRE team. Use labels to differentiate between "Chaos Testing" and "Production Incident."

Integrating chaos testing into CI/CD pipelines

Integrating chaos testing into CI/CD pipelines

Integrating Chaos Engineering into your CI/CD pipelines transforms resilience from a periodic audit into a continuous verification process. By automating chaos experiments, you ensure that as your infrastructure, configurations, and application code evolve, your system's ability to withstand failure remains intact.

Why integrate into CI/CD?

Software systems are dynamic. A configuration change, a dependency update, or a new version of an operator can introduce subtle failure modes. Continuous testing in your pipelines provides:

- **Regression testing for resilience:** Catch unintended side effects where a service recovery path is broken by new code.
- **Early feedback:** Detect failures in deployment manifests or readiness probes before they reach higher environments.
- **Standardized execution:** Ensure that the same rigorous fault injection criteria are applied consistently across every release candidate.

Pipeline Integration Strategy

To effectively integrate Krkn into your pipeline, follow these best practices:

1. **Define the Scope:** Do not run your entire suite of experiments on every commit. Use chaos testing for specific stages (e.g., integration, performance testing, or nightly builds).
2. **Ensure Observability:** The pipeline should be connected to your monitoring stack (e.g., Prometheus/Grafana) to validate the "Steady State" before, during, and after the experiment.
3. **Automated Rollback:** Always configure the pipeline to trigger an immediate rollback or recovery action if the chaos experiment causes an unacceptable breach of your Service Level Objectives (SLOs).

Hands-on Activity: Automating a Pod Disruption Experiment

In this activity, you will integrate a basic Krkn pod-disruption scenario into an automated pipeline step.

Prerequisites

- A running cluster with the Krkn Operator installed.
- Access to your CI/CD runner (e.g., Tekton, GitHub Actions, or Jenkins).
- A configured `scenario.yaml` file defining the target namespace and disruption type.

Step 1: Configure the experiment manifest

Create a minimal `pod-disruption.yaml` that identifies the target application.

```
chaos_engine:
  scenario: pod-disruption
  target_namespace: "my-app-prod"
  duration: 60
  label_selector: "app=backend-api"
  kill_count: 2
```

Step 2: Define the CI Pipeline Task

Add a step to your pipeline to execute the Krkn job. Using a shell-based CI runner, the task should trigger the execution and wait for the status report.

```
# Example pipeline script
echo "Starting Chaos Experiment..."
# Trigger the Krkn job via oc/kubectl
oc apply -f pod-disruption.yaml -n krkn-chaos

# Poll for experiment completion
while [ $(oc get chaosengine -n krkn-chaos -o jsonpath='{.items[0].status.status}') !=
"Completed" ]; do
  echo "Waiting for chaos experiment to finish..."
```

```
sleep 10
done

# Validate against resilience metrics
./scripts/validate-slo.sh --threshold 99.9
```

Step 3: Implement Post-Experiment Validation

The pipeline must not just execute the chaos; it must verify recovery. Include a script that checks: * **Pod Readiness:** Are all replicas back to a 'Running' state? * **SLO adherence:** Did the latency remain within defined bounds during the experiment window?

```
# Check if pods are ready
READY_PODS=$(oc get deployment backend-api -n my-app-prod -o
jsonpath='{.status.readyReplicas}')
EXPECTED_PODS=3

if [ "$READY_PODS" -eq "$EXPECTED_PODS" ]; then
    echo "Recovery Successful: All pods are ready."
else
    echo "Recovery Failed: Expected $EXPECTED_PODS, got $READY_PODS"
    exit 1
fi
```

Best Practices for Pipeline Success

- **Fail Fast:** If the baseline health check fails before the experiment starts, abort the pipeline immediately. Do not inject chaos into an already unstable system.
- **Log Collection:** Configure the pipeline to archive the Krkn logs and experiment reports as build artifacts for auditability and historical analysis.
- **Isolation:** Use dedicated namespaces for chaos execution to ensure that the blast radius remains contained within the scope of the test.

Running chaos during performance and scale testing

Running chaos during performance and scale testing

In distributed systems, testing performance and scalability in isolation often leads to a false sense of security. While your application might handle high traffic loads under ideal conditions, it may fail catastrophically when infrastructure instability is introduced. Integrating chaos engineering directly into performance and scale testing allows you to observe how your system's "Steady State"—typically defined by latency, throughput, and error rates—degrades under combined stress.

Why combine chaos with performance testing?

Performance testing validates that your system meets throughput and latency requirements under load. Adding chaos testing to these runs provides critical insights:

- **Resource Contention:** Identifies if specific components become bottlenecks when resources (CPU/Memory) are throttled during high load.
- **Failover Efficacy:** Observes if automated recovery mechanisms (e.g., pod restarts, service migrations) actually trigger and succeed when the system is already under resource pressure.
- **Cascading Failure Analysis:** Determines if a minor disruption in one service causes a systemic collapse under heavy load.
- **Signaling and Coordination:** Uses tools like Krkn's signaling feature to ensure chaos events are synchronized with the ramp-up or peak periods of your load tests.

Implementing integrated testing

To effectively run chaos during scale testing, follow these architectural principles:

1. Define the Baseline

Before introducing faults, establish your performance baseline. Use PromQL to define your Service Level Objectives (SLOs): * **Latency:** P99 response times during peak load. * **Throughput:** Requests per second (RPS) handled by the ingress. * **Error Rates:** Percentage of 5xx HTTP codes.

2. Managing Chaos Lifecycle with Load

Use Krkn to orchestrate the chaos events in coordination with your load generation tool (e.g., Locust, k6, or Jaeger).

```
# Example structure for coordinating chaos with load testing
chaos_scenario:
  type: pod-disruption
  target: "auth-service"
  start_after_load_reaches: 80% # Using signaling/wait conditions
  duration: 5m
```

3. Observing Impact

During the execution, monitor if the performance metrics deviate significantly from the baseline defined in step 1. If the system fails to recover or the performance degradation exceeds the SLO threshold, trigger an automated stop condition.

Hands-on Lab: Synchronizing Chaos with Load

In this exercise, you will run a pod-disruption experiment while your application is under a synthetic load to measure resilience.

1. Prerequisites

- A cluster with a deployed application and Krkn Operator installed.
- A load testing tool configured to hit your application ingress.

- Step 1: Start the Load Generator** Begin your load test and wait for the system to reach a steady-state at 70% of peak capacity.

- Step 2: Define the Krkn Scenario** Create a `chaos-config.yaml` focusing on a specific microservice. Use a moderate blast radius to ensure you are testing the resiliency of the application logic, not just infrastructure collapse.

```
scenario:
  name: "Auth-Service-Load-Test-Disruption"
  type: "pod-disruption"
  target_pods: "auth-service-*"
  duration: 300
  percentage_to_kill: 25
```

- Step 3: Execute and Monitor** Execute the scenario via the Krkn operator:

```
krkn-operator run --config chaos-config.yaml
```

- Step 4: Analyze the Resilience Score** Compare the P99 latency recorded during the chaos period against your baseline.
 - **Observation:** Did the load balancer correctly reroute traffic?
 - **Validation:** Did the application recover automatically once the Krkn experiment concluded, or did it require manual intervention?

Best Practices for Integrated Testing

- **Stop Conditions:** Always define clear thresholds (e.g., "if error rate > 5%, abort chaos"). This prevents total outage during a performance test.
- **Document Dependencies:** Ensure that your metrics monitoring includes the health of underlying infrastructure (nodes/nodes-network) to distinguish between application issues and resource starvation.
- **Iterate:** Start with minor disruptions (single pod deletion) during low load, then increase intensity and concurrency as you gain confidence in your system's recovery models.

Standardizing execution across managed clusters

Standardizing execution across managed clusters

In large-scale Kubernetes environments, running isolated chaos experiments is rarely sufficient. To achieve true platform resiliency, organizations must transition from ad-hoc testing to a standardized model where chaos experiments are applied consistently across managed clusters. This ensures that infrastructure, security policies, and application availability remain predictable regardless of which cluster hosts the workload.

The Need for Standardization

Standardizing chaos execution across managed clusters addresses several critical operational challenges:

- **Consistency:** Ensuring that all clusters meet the same baseline resiliency standards (e.g., verifying that pod disruptions in cluster A trigger the same recovery mechanisms as in cluster B).
- **Compliance:** Providing auditable proof that recovery validation has been performed across the entire fleet.
- **Efficiency:** Reducing the "toil" associated with configuring repetitive experiments by utilizing centralized registries and reusable workflow templates.
- **Blast Radius Control:** Applying standardized stop-conditions and rollback thresholds to prevent local experiments from escalating into global outages.

Strategies for Uniform Execution

1. Centralized Scenario Registries

Rather than defining scenarios locally, leverage centralized scenario registries. By storing your disruption configurations in a shared repository:

- * Teams can pull vetted, "approved" chaos scenarios.
- * Version control is enforced, ensuring that updates to a disruption method are pushed across all clusters simultaneously.
- * Configurations for parameters—such as target duration, resource saturation levels, or chaos interval—remain consistent across staging, QA, and production.

2. Multi-cluster Discovery and Targeting

Using Krkn's integration with ACM (Advanced Cluster Management) and OCM (Open Cluster Management), you can categorize clusters into logical groups (e.g., `prod-us-east`, `prod-eu-west`). When executing an experiment:

- * Define a target group rather than individual nodes.
- * Execute the scenario across all selected clusters simultaneously to observe global impact.
- * Use standard tagging to ensure experiments only run on clusters that meet specific architectural prerequisites (e.g., node sizing or specific resource reservation policies).

3. Standardizing Recovery Validation (PromQL)

To standardize how success is measured, move away from subjective observation to standardized PromQL queries stored within the Krkn file management system.

- * **Define Global SLOs:** Use consistent PromQL queries across all clusters to measure availability, latency, and throughput during a chaos event.
- * **Resiliency Scores:** Normalize the "Resiliency Score" calculation so that an experiment in a dev cluster and a prod cluster produce comparable data points.

Hands-on Activity: Deploying a Standardized Experiment

In this activity, you will execute a standardized pod-disruption scenario across multiple managed clusters using the Krkn interface.

1. **Step 1: Select Target Group** Navigate to the **Target Clusters** menu. Select your clusters based on the defined labels (e.g., `environment: production`). Ensure that the clusters are connected and reporting status.
2. **Step 2: Load the Standardized Scenario** Go to the **Scenario Registry**. Select the pre-configured `pod-disruption-standard` template. This template contains the pre-approved duration, target namespaces, and jitter settings required by your organization's compliance

policy.

3. Step 3: Define Execution Parameters

- Set the `blast-radius` parameter to **15%** of total replicas.
- Attach the standardized PromQL query file `standard-slo-check.promql` to the experiment configuration.

4. **Step 4: Execute and Observe** Click **Execute**. Monitor the experiment progress via the **Jobs** tab. Because this is a multi-cluster execution, observe the log streaming from all target clusters in a unified dashboard view.

5. **Step 5: Compare Results** Once the experiment completes, navigate to the **Resiliency Score** dashboard. Compare the scores of your target clusters.

- **Observation:** If one cluster shows a significantly lower score, investigate if it deviates from the standard node sizing or resource reservation best practices identified in your system baseline.

Best Practice: Guardrails for Standardization

To maintain stability while standardizing: * **Hard-code Stop Conditions:** Always include a mandatory stop-condition in your standard workflow template (e.g., `if latency > 500ms, abort experiment`). * **Immutable Configurations:** Once a workflow is validated in a test environment, promote it to a read-only registry for production use. * **Audit Logging:** Ensure that all executions across managed clusters are logged with the user ID, timestamp, and the specific version of the registry used.

Storing and comparing long-term results

Storing and comparing long-term results

In a mature Chaos Engineering practice, a single experiment provides a snapshot of system health, but long-term data collection provides the "resiliency trajectory." By storing and comparing results over time, you can determine if your platform is becoming more robust or if architectural changes are introducing regression.

The Importance of Historical Data

Chaos experiments should not be treated as ephemeral events. Storing results serves three primary operational needs:

- **Trend Analysis:** Identifying if resilience scores are trending upward (improvements) or downward (technical debt accumulation).
- **Regression Detection:** Validating that infrastructure upgrades, patch management, or configuration changes have not inadvertently degraded fault tolerance.
- **Compliance and Audit:** Providing documented evidence that the system meets Service Level Objectives (SLOs) under adverse conditions.

Strategies for Long-Term Storage

To effectively analyze long-term results, consider these architectural approaches for data persistence:

1. **Centralized Result Aggregation:** Use the Krkn Jobs output data to feed into a time-series database (such as Prometheus or Thanos). Ensure that job metadata—such as cluster IDs, scenario versions, and timestamps—is captured as labels.
2. **Versioned Workflow Models:** Keep your Chaos Studio workflow definitions in version control (Git). Linking a specific experiment run to a Git commit hash allows you to correlate system performance directly with infrastructure-as-code changes.
3. **Result Normalization:** Since Krkn produces detailed logs and metrics, standardize the output format to ensure that comparisons between "Run A" (three months ago) and "Run B" (today) remain statistically valid.

Comparing Results Over Time

When evaluating performance trends, focus on the following comparative metrics:

- **Recovery Time Objective (RTO) Drift:** Monitor if the time taken for services to recover from a specific disruption scenario is increasing.
- **Resiliency Score Variance:** Track the scores generated by the Resiliency Score feature. A stable score despite increased traffic or complexity indicates successful architectural hardening.
- **Error Rate Correlation:** Compare the error rates during chaos events across different months. Decreasing peaks during the same disruption type is a primary indicator of improved resilience.

Hands-on Lab: Analyzing Historical Resilience

This lab demonstrates how to track improvement by comparing results from two different experimental runs.

Prerequisites

- Access to a Krkn environment with at least two completed chaos workflows.
- PromQL queries defined for your application's SLIs.

Step 1: Exporting Historical Run Data

1. Navigate to the **Jobs** page in the Krkn dashboard.
2. Identify two runs of the same scenario (e.g., **pod-disruption**) executed at different time intervals.
3. Export the JSON-formatted report for both runs.

Step 2: Comparing Resilience Scores

Using the Krkn CLI or your observability tool of choice, pull the specific Resiliency Score values:

```
# Example: Retrieving scores for specific job IDs
krkn get-resilience-score --job-id <JOB_ID_OLD>
```

```
krkn get-resilience-score --job-id <JOB_ID_NEW>
```

Step 3: Mapping Trends to Changes

Create a simple comparison table to visualize the improvement:

Metric	Run (Jan 1)	Run (Mar 1)	Change	:---	:---	:---	:---	Resiliency Score	72	88
+16	Recovery Duration	120s	85s	-35s	Error Spikes	5%	1%	-4%		

Self-Check: Analyze if the improvement correlates with a specific infrastructure update or code deployment made between the two dates. If the score dropped, investigate which dependent components experienced increased latency during the latter experiment.

Troubleshooting Tips for Long-Term Comparisons

- **Consistent Baselines:** If your baseline traffic changes, your comparison becomes invalid. Ensure experiments are run under similar load conditions to maintain "apples-to-apples" comparisons.
- **Scenario Drift:** If you update a scenario (e.g., increasing the number of replicas killed), ensure you tag the experiment with a version identifier so that the comparison takes the change in intensity into account.

Combining recovery checks with system health metrics

Combining recovery checks with system health metrics

In Chaos Engineering, verifying that a specific component has recovered after a disruption is only the first step. To truly understand system resilience, you must correlate component recovery with the overall health of the cluster and application performance metrics. Krkn achieves this through a multi-layered validation strategy that combines localized recovery checks, holistic cluster health signals (via Cerberus), and custom PromQL metric evaluations.

The Triple-Layer Validation Strategy

To ensure a comprehensive assessment of system resilience, Krkn uses three distinct validation layers:

1. **Component Recovery Checks:** Most Krkn scenarios include built-in logic to verify that the targeted resources (e.g., pods, nodes) have returned to a "Ready" state after the chaos duration.
2. **Holistic Cluster Health (Cerberus):** A global health signal that monitors the entire environment. If Cerberus reports an unhealthy state, Krkn can halt the experiment to prevent further collateral damage.
3. **Metrics Evaluation (PromQL):** Evaluating specific performance indicators—such as latency, throughput, or etcd health—against defined Service Level Objectives (SLOs) to determine the true impact of the disruption.

Evaluating System Health via PromQL

Beyond basic "up/down" status, performance metrics provide the context needed to identify silent failures or degraded user experiences. Krkn allows you to query the in-cluster Prometheus instance to validate system stability during and after chaos.

You can define custom PromQL expressions that return an exit status of **0** (pass) or **1** (fail) based on the severity of the metrics observed. This is critical for catching issues such as: * **Resource Spikes:** Unexpected CPU or memory pressure during recovery. * **Etcd Health:** Monitoring disk fsync latency or leader elections that could lead to cluster instability. * **Application Throughput:** Ensuring that while the component is "running," it is actually processing the expected volume of transactions.

Implementing Recovery Validation: A Hands-On Approach

In this exercise, we will configure a Krkn experiment to validate both the recovery of a targeted component and the overall health of the system using PromQL.

Prerequisites

- A running OpenShift cluster with Krkn and Cerberus installed.
- Permissions to modify **ChaosScenario** definitions.

Step 1: Defining the Metrics Evaluation

Create a configuration file to evaluate the **etcd_disk_wal_fsync_duration_seconds** to ensure that cluster storage performance remains within acceptable bounds during the experiment.

```
# prometheus_evaluation.yaml
metrics:
  - name: "Check etcd fsync latency"
    query: "histogram_quantile(0.99,
sum(rate(etcd_disk_wal_fsync_duration_seconds_bucket[5m])) by (le))"
    threshold: 0.01 # Acceptable latency limit
    severity: "critical"
    operator: "<"
```

Step 2: Configuring the Experiment

Integrate the health checks into your chaos scenario. This ensures that the scenario considers both the component recovery and the custom metric evaluation as part of its pass/fail criteria.

```
# chaos_experiment.yaml
scenario:
  type: "pod-disruption"
  target: "my-app-deployment"
  # Recovery check: built-in
  duration: 300
  # Holistic check: Cerberus
```

```
cerberus_enabled: true
# Metric evaluation: In-cluster PromQL
metrics_check:
  enabled: true
  config_file: "prometheus_evaluation.yaml"
```

Step 3: Executing and Analyzing

1. Execute the scenario using the Krkn CLI or via the Chaos Studio workflow.
2. Monitor the logs for the `metrics_evaluation` phase.
3. If the scenario fails, check the logs to see if the failure was triggered by the component failing to restart or by the PromQL metric exceeding the `threshold`.



Best Practice: Always deploy Machine Health Checks in your cluster. They act as the primary remediation layer, ensuring that unhealthy nodes are replaced automatically. This provides a baseline for stability, allowing Krkn to measure whether your **application** logic is resilient even when the infrastructure recovers correctly.

Summary Checklist for Validation

- ☐ Does the scenario include internal recovery checks for the targeted pods/nodes?
- ☐ Is Cerberus configured to signal the overall health of the cluster?
- ☐ Are there PromQL queries configured to catch performance degradation that doesn't trigger a hard "down" status?
- ☐ Are exit conditions clearly mapped to the severity of the violation?

Hands-on Labs

Hands-on Labs

The hands-on labs provide a practical environment to apply chaos engineering principles using the Krkn ecosystem. These labs focus on transitioning from theoretical risk identification to active system validation through the following activities:

- **Experiment Execution:** Performing controlled pod-disruption tests to observe system behavior and recovery patterns.
- **Workflow Modeling:** Creating and running multi-stage failure scenarios using Chaos Studio to simulate complex outages.
- **Resilience Analysis:** Utilizing PromQL and telemetry data to calculate and interpret Resilience Scores, establishing a quantitative baseline for system health.
- **Automation:** Integrating chaos scenarios into CI/CD pipelines to standardize resilience validation across test environments.

Executing pod-disruption experiments

Executing Pod-Disruption Experiments

Pod-disruption experiments are the foundational building blocks of Chaos Engineering in Kubernetes. By intentionally terminating pods or disrupting their lifecycle, you can validate the self-healing capabilities of your applications, verify readiness/liveness probes, and ensure that your services maintain availability during transient failures.

Understanding Pod Disruptions

Pod disruptions typically simulate common failure modes such as:

- * **Unexpected crashes:** Simulating a process failure or OOM (Out of Memory) event.
- * **Scheduling delays:** Understanding how long it takes for a replacement pod to become "Ready."
- * **Service Availability:** Ensuring that traffic is correctly routed away from terminating pods to healthy replicas.

Prerequisites

Before starting, ensure you have:

- * A Kubernetes cluster (target cluster).
- * `krknctl` installed, or the Krkn Operator deployed.
- * Access to the `krkn-hub` image repository.

Hands-on Lab: Executing a Pod-Deletion Experiment

In this lab, we will use `krknctl` to execute a controlled pod-deletion scenario. This will terminate pods matching specific labels to observe how the deployment controller handles the replacement and service traffic.

Step 1: Identify the Target

Identify the deployment you wish to test. For this exercise, we will target a dummy application in the `chaos-test` namespace.

```
# List pods to find your target
kubectl get pods -n chaos-test
```

Step 2: Configure the Scenario

Pod scenarios in Krkn allow you to define the `label_selector` to ensure you only disrupt the intended workload. Create a configuration file (or define environment variables) for the `pod-scenarios`.

```
# example-pod-chaos.yaml
scenario: pod-scenarios
label_selector: "app=frontend"
namespace: "chaos-test"
duration: 60s
kill_count: 2
```

Step 3: Execute the Experiment

Using `krknctl`, trigger the chaos experiment. This abstracts the container runtime complexities and initiates the disruption.

```
krknctl run pod-scenarios \
  --config example-pod-chaos.yaml
```

Step 4: Observability and Validation

While the experiment is running, monitor the behavior of the targeted deployment in a separate terminal:

```
# Monitor the deployment rollout
kubectl get pods -n chaos-test -w
```

Look for the following indicators: 1. **Termination:** Verify that pods enter the `Terminating` state. 2. **Re-creation:** Verify that the ReplicaSet immediately triggers the creation of new pods. 3. **Recovery Time:** Measure the duration from the start of the `Terminating` signal until the new pod reaches the `Running` and `Ready` state.

Step 5: Post-Experiment Analysis

Once the execution finishes, inspect the logs generated by Krkn to verify the successful impact of the experiment.

```
# View scenario status reports
krknctl status --job <job-id>
```

Troubleshooting Common Issues

- **Pods not terminating:** Verify that the `label_selector` matches the pods exactly. If using an incorrect key/value pair, the experiment may report 0 pods affected.
- **Permission Denied:** Ensure that the `ServiceAccount` used by the Krkn Operator or `krknctl` has sufficient RBAC permissions (specifically `delete` and `list` verbs for the `pods` resource).
- **Metrics Missing:** If monitoring via Prometheus/Grafana, ensure your `steady-state` metrics are being scraped at a high enough frequency to capture the rapid pod replacement cycle.

Summary

By executing pod-disruption experiments, you move beyond theory and into evidence-based reliability. You have now learned how to target specific pods, trigger controlled deletions, and observe the cluster's orchestration response. In subsequent sections, we will build upon this by chaining these disruptions into complex, multi-stage failure workflows.

Modeling multi-stage failure workflows

Modeling multi-stage failure workflows

In complex distributed systems, failures rarely happen in isolation. A cascading failure—where one component's degradation triggers a chain reaction—is often more damaging than a single node outage. Modeling **multi-stage failure workflows** allows you to simulate these complex, real-world failure patterns to validate that your system's observability, self-healing mechanisms, and fallback strategies function as expected.

Anatomy of a Multi-Stage Workflow

A multi-stage workflow in Chaos Studio is a directed graph of disruption scenarios. By chaining these scenarios, you move beyond simple "pod kill" tests to simulate sophisticated incidents such as:

- **Dependency Cascades:** Disrupting a downstream service (e.g., a database) followed by an upstream service (e.g., a cache layer).
- **Resource Contention:** Injecting latency into a network segment, followed by a memory leak simulation on the nodes serving that segment.
- **Recovery Validation:** Injecting a failure, pausing to allow for automated remediation, and then injecting a second failure to see if the system remains resilient under cumulative pressure.

Designing Workflows in Chaos Studio

Chaos Studio provides a visual drag-and-drop editor to define these sequences. You can configure nodes to execute in two primary patterns:

- **Sequential Execution:** Scenarios execute one after another. Stage B begins only after Stage A completes (or after a defined delay). This is ideal for testing how a system recovers from one failure before another is introduced.
- **Parallel Execution:** Multiple disruption scenarios occur simultaneously. This is used to test "storm" scenarios, such as simultaneous high CPU load across multiple zones coupled with network partitioning.

Hands-on Lab: Modeling a Cascading Failure

This lab guides you through the creation of a two-stage workflow that simulates a service degradation followed by a partial network outage.

Prerequisites

- Access to the Krkn Operator dashboard.
- Two defined disruption scenarios: `pod-memory-leak` and `network-latency`.
- A target namespace/cluster configured in your Registry.

Steps to Create the Workflow

1. **Initialize the Canvas:** Navigate to **Chaos Studio** and create a new workflow titled `cascading-failure-simulation`.
2. **Add Stage 1 (Memory Pressure):**
 - Drag the `pod-memory-leak` scenario onto the canvas.
 - Configure the duration (e.g., 5 minutes) and the target labels.
 - Set the "Blast Radius" to specific worker nodes to ensure local impact.
3. **Add Stage 2 (Network Latency):**
 - Drag the `network-latency` scenario onto the canvas.
 - Connect the output of the first node to the input of the second node.
 - Configure the latency parameters (e.g., 500ms delay) to target the same application pods affected by the memory leak.
4. **Configure Orchestration:**
 - Select the connector between the two nodes.
 - Choose **Sequential** execution.
 - Add a "Wait" step (e.g., 60 seconds) between the nodes to verify if the application attempted a recovery between stages.
5. **Enable Resiliency Scoring:**
 - Select the workflow settings.
 - Enable the **Resiliency Score**.
 - Attach your pre-defined `slos.promql` file to monitor throughput and error rates during both stages of the experiment.

6. Execute and Observe:

- Click **Run**.
- Monitor the **Jobs** tab. Observe how the Resiliency Score fluctuates as the second stage of the workflow initiates.

Troubleshooting Tips

- **Check Stop Conditions:** If your system hits a critical SLO threshold early, ensure you have defined "Stop Conditions" in the workflow settings to halt the experiment and prevent a total outage.
- **Analyze the Logs:** If a stage fails to start, verify the **target-cluster** permissions in the Krkn Operator to ensure it has the necessary RBAC to execute the requested disruption on the target pods.
- **Review Recovery:** After the workflow ends, check if your **machine health checks** successfully remediated the nodes or if manual intervention was required. Use the discrepancy between expected and actual recovery time to refine your automation settings.

Calculating and analyzing Resilience Scores

Calculating and analyzing Resilience Scores

The Resiliency Score is a quantitative metric provided by Krkn that moves beyond binary pass/fail outcomes. It offers a nuanced view of how well your application maintains its service level objectives (SLOs) when subjected to turbulent conditions. By evaluating real-time application telemetry during a chaos workflow, the Resiliency Score provides a baseline to track architectural hardening over time.

Understanding the Scoring Mechanism

The Resiliency Score is derived from custom PromQL queries that define the "health" of your application. Instead of focusing solely on the disruption itself, the score focuses on the **impact** of that disruption on your defined steady state.

The Evaluation Lifecycle

1. **Metric Definition:** You define a set of PromQL queries that represent your key performance indicators (KPIs), such as request latency, error rates, or successful transaction throughput.
2. **Threshold Setting:** Each metric is paired with a threshold that defines the "Acceptable" boundary.
3. **Real-time Monitoring:** During the execution of a chaos workflow, Krkn continuously samples these metrics.
4. **Aggregation:** The Resiliency Score is calculated by aggregating the duration and magnitude of time the application spends outside of these defined thresholds relative to the total duration of the chaos experiment.

Configuring Resiliency Scoring

To enable scoring, you must provide Krkn with the necessary observability configuration via the File Management system.

1. **Define the PromQL file:** Create a configuration file containing the queries used to evaluate system behavior.
2. **Enable in Workflow:** Within your Chaos Studio workflow, toggle the **Resiliency Score** option.
3. **Mount the File:** Attach your PromQL file to the workflow execution.

```
# Example structure of a resiliency-score-config.yaml
metrics:
  - name: "http_error_rate"
    query: "sum(rate(http_requests_total{status=~'5..'}[1m])) /
sum(rate(http_requests_total[1m]))"
    threshold: 0.05 # 5% error rate threshold
  - name: "p99_latency"
    query: "histogram_quantile(0.99,
sum(rate(http_request_duration_seconds_bucket[1m])) by (le))"
    threshold: 0.5 # 500ms latency threshold
```

Analyzing and Interpreting Results

Once the workflow completes, the **Jobs** page displays the final numeric score.

- **100% (or equivalent scale):** Indicates that the application maintained its steady state perfectly throughout the disruption.
- **Lower Scores:** Indicate that the service level indicators (SLIs) deviated from the expected thresholds for a significant portion of the test.

Remediation Strategy

Use the score analysis to drive infrastructure and application improvements:

- * **Trend Identification:** Compare scores across multiple runs. A decreasing score after a deployment suggests a regression in resiliency.
- * **Threshold Tuning:** If the score is consistently low but the application feels stable, evaluate if your PromQL thresholds are too aggressive for the current architecture.
- * **Component Isolation:** If a workflow involves multiple stages, use the graph run results to identify which specific disruption scenario caused the largest dip in the Resiliency Score, highlighting a specific dependency that needs hardening.

Hands-on Lab: Calculating your First Score

In this lab, you will evaluate the resiliency of a sample frontend service.

Prerequisites:

- * An active cluster with Krkn Operator installed.
- * Access to the Chaos Studio dashboard.

Steps:

1. **Define Metrics:** Create a `resiliency.yaml` file targeting your `frontend-service` error rate and latency. Upload this to the Krkn File Management system.
2. **Create Workflow:**
 - Navigate to **Chaos Studio**.
 - Select a "Pod Disruption" scenario.
 - Enable the **Resiliency Score** toggle.
 - Select your uploaded `resiliency.yaml` file from the dropdown.
3. **Execute:** Run the workflow. Ensure you have the application under light load to generate sufficient metrics.
4. **Evaluate:**
 - Once finished, navigate to the **Jobs** page.
 - Click on the completed run to view the Score.
 - Compare the latency graph against the threshold lines.
 - **Question:** Did the latency spike correlate exactly with the pod termination, or did the recovery process introduce additional performance degradation?
5. **Refine:** Based on the result, adjust your pod autoscaling (HPA) or increase replica counts and re-run the experiment to see if the score improves.

Automating resilience validation in test environments

Automating resilience validation in test environments

As systems grow in complexity, manual chaos testing becomes unsustainable. Automating resilience validation allows you to move from "point-in-time" testing to continuous verification. By integrating Krkn scenarios into your CI/CD pipelines, you ensure that infrastructure or application changes do not introduce regressions in system stability.

The Role of Continuous Resilience

Automating resilience means treating "Chaos as Code." When a deployment triggers a test pipeline, the pipeline should not only verify functional requirements but also validate the system's ability to recover from simulated failures.

Key benefits of automation include: * **Consistency:** Every release is subjected to the same baseline of stress testing. * **Early Detection:** Discovering circular dependencies or inadequate timeout configurations during the PR stage rather than in production. * **Feedback Loops:** Automatically generating Resiliency Scores for every build to track degradation over time.

Integrating Krkn into CI/CD

To automate Krkn, you move away from manual dashboard triggers and utilize the Krkn API or CLI-driven execution within your pipeline runners (e.g., Jenkins, Tekton, or GitHub Actions).

Pipeline Integration Strategy

1. **Trigger:** The CI/CD pipeline deploys the target application to a pre-production environment.
2. **Setup:** The pipeline triggers a Krkn scenario via the command line or an API call.
3. **Execution:** Krkn injects the disruption (e.g., killing pods, inducing network latency).
4. **Monitoring/Wait:** The pipeline waits for a signal that the recovery phase has concluded.
5. **Evaluation:** The system queries PromQL metrics to evaluate the "Steady State" and compares the results against the established Resilience Score.
6. **Gate:** If the Resilience Score falls below the acceptable threshold, the pipeline marks the build as failed.

Hands-on Lab: Automating a Pod-Disruption Gate

In this lab, you will configure a simple pipeline step to execute a pod disruption and fail the build if the system does not recover within the expected timeframe.



Ensure you have the **krkn** CLI tool installed and your **kubeconfig** pointing to your test environment.

Step 1: Define the Scenario Configuration

Create a file named **pod-failure-ci.yaml**. This configuration dictates the blast radius and the duration of the test.

```
scenario:
  name: "automated-pod-failure"
  duration: 60
  instance_count: 3
  label_selector: "app=frontend"
```

Step 2: Execute and Validate

In your pipeline script (e.g., **Jenkinsfile** or **task.yaml**), call Krkn as part of the validation stage.

```
# Trigger the chaos experiment
krkn run pod-disruption -c pod-failure-ci.yaml --report-dir=./reports

# Analyze the results
# Here we check if the system recovered successfully
if grep -q "recovered" ./reports/status.log; then
  echo "Resilience validation passed."
else
  echo "Resilience validation failed: System did not recover."
  exit 1
fi
```

Step 3: Verify Metrics with PromQL

To fully automate the gate, use the Krkn PromQL integration to query the steady state during the experiment. Add this check to your automation script to ensure availability remained above 99% during the disruption:

```
# PromQL check for error rate
query="sum(rate(http_requests_total{status=~'5..'}[1m])) /
sum(rate(http_requests_total[1m])) < 0.01"

# Execute check against Prometheus API
result=$(curl -G 'http://prometheus-url/api/v1/query' --data-urlencode "query=$query")

# If result is empty or false, fail the build
```

Best Practices for Automated Resilience

- **Small Blast Radius:** In CI/CD, keep the blast radius limited to the service being tested to avoid cascading failures in shared test environments.
- **Short Durations:** Automated tests should be fast. Focus on whether the service **recovers**, not how long it can stay in a degraded state.
- **Stop Conditions:** Always configure "Kill Switches" in your automation. If critical infrastructure metrics (e.g., node pressure) hit a defined limit, the automation must abort the chaos scenario immediately to prevent permanent environment damage.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)